```
// ======== 文件：src/main.cpp ========
// -*- coding: utf-8 -*-
// main.cpp —— cpp_sim.exe 入口。
//
//   cpp_sim --selfcheck
//   cpp_sim --params <file>            跑一次模拟（params.txt 见 README）
//   cpp_sim --dump-interp <in.csv> <out.csv>   PCHIP 探针（供与 scipy 对比）
//   cpp_sim --dump-wetbulb <out.csv>
//   cpp_sim --dump-props <out.csv>
//
// 退出码：0 = 成功；1 = 求解/断言失败；2 = 配置错误。
//
// ⚠ 用 wmain（-municode）拿宽字符命令行：`H:\2026 数学建模\` 含中文，MinGW 的
//   窄字符 argv 会按 ANSI 码页解码而乱码。见 params.hpp 的说明。
#include <cstdio>
#include <iostream>
#include <string>
#include <vector>

#include "interp.hpp"
#include "params.hpp"
#include "props.hpp"
#include "psychro.hpp"
#include "run.hpp"
#include "util.hpp"

int run_selfcheck();   // selfcheck.cpp

namespace {

std::string narrow(const wchar_t* w) {
    std::string s;
    for (const wchar_t* p = w; *p; ++p) {
        if (*p > 127) {
            throw util::ConfigError(
                "命令行参数含非 ASCII 字符；请只传 ASCII 相对路径");
        }
        s.push_back(static_cast<char>(*p));
    }
    return s;
}

// ---------------------------------------------------------------------------
// 探针：PCHIP
// ---------------------------------------------------------------------------
int dump_interp(const std::string& in, const std::string& out) {
    io::Table t;
    t.load(in);
    if (t.ncols() < 2) {
        throw util::ConfigError("--dump-interp 需要至少 2 列: " + in);
```

```
    }
    interp::Pchip pT(t.xvec(), t.col(0));
    interp::Pchip pC(t.xvec(), t.col(1));
    io::CsvWriter w(out, "t,Pchip_col1,Pchip_col2");
    for (const auto& xi : t.xvec()) {
        std::vector<double> row{xi, pT(xi), pC(xi)};
        w.row(row);
    }
    // 节点之间也打一圈（等距 5000 点）
    const double a = t.xvec().front(), b = t.xvec().back();
    const int M = 5000;
    for (int i = 0; i <= M; ++i) {
        const double x = a + (b - a) * i / M;
        std::vector<double> row{x, pT(x), pC(x)};
        w.row(row);
    }
    w.close();
    std::printf("DONE dump-interp rows=%d out=%s\n", M + 1 + (int)t.nrows(),
                out.c_str());
    return 0;
}

// ---------------------------------------------------------------------------
// 探针：湿球温度
// ---------------------------------------------------------------------------
int dump_wetbulb(const std::string& out) {
    io::CsvWriter w(out, "T_inf_C,C_inf,Tw_C");
    for (int i = 0; i <= 20; ++i) {
        const double Ta = 20.0 + 2.0 * i;                  // 20 … 60 ℃
        for (int j = 0; j <= 20; ++j) {
            const double Ci = 0.005 + 0.0025 * j;          // 0.005 … 0.055
            std::vector<double> row{Ta, Ci, psychro::wet_bulb(Ta, Ci, 25.0)};
            w.row(row);
        }
    }
    w.close();
    std::printf("DONE dump-wetbulb out=%s\n", out.c_str());
    return 0;
}

// ---------------------------------------------------------------------------
// 探针：物性
// ---------------------------------------------------------------------------
int dump_props(const std::string& out) {
    auto p2 = phys::props_a();
    auto p3 = phys::props_b();
    auto p4 = phys::props_c();
    io::CsvWriter w(out,
                    "C,T_K,rho2,cp2,k2,D2,rho3,cp3,k3,D3,rho4,cp4,k4,D4");
    for (int i = 0; i <= 100; ++i) {
```

```
        const double C = 0.02 + (3.0 - 0.02) * i / 100.0;
        for (int j = 0; j <= 10; ++j) {
            const double T = 273.15 + 20.0 + 4.0 * j;      // 20 … 60 ℃
            std::vector<double> row{
                C, T,
                p2->rho(C), p2->cp(C), p2->k(C), p2->D(C, T),
                p3->rho(C), p3->cp(C), p3->k(C), p3->D(C, T),
                p4->rho(C), p4->cp(C), p4->k(C), p4->D(C, T)};
            w.row(row);
        }
    }
    w.close();
    std::printf("DONE dump-props out=%s\n", out.c_str());
    return 0;
}

int real_main(const std::vector<std::string>& args) {
    if (args.size() < 2) {
        std::printf("cpp_sim —— 药材烘干过程数值模拟软件  核心求解器\n");
        std::printf("  --selfcheck\n");
        std::printf("  --params <file>\n");
        std::printf("  --dump-interp <in.csv> <out.csv>\n");
        std::printf("  --dump-wetbulb <out.csv>\n");
        std::printf("  --dump-props <out.csv>\n");
        return 0;
    }
    const std::string cmd = args[1];
    if (cmd == "--selfcheck") return run_selfcheck();
    if (cmd == "--params") {
        if (args.size() < 3) throw util::ConfigError("--params 需要文件路径");
        return runner::run_from_params(args[2]);
    }
    if (cmd == "--dump-interp") {
        if (args.size() < 4) throw util::ConfigError("--dump-interp 需要两个路径");
        return dump_interp(args[2], args[3]);
    }
    if (cmd == "--dump-wetbulb") {
        if (args.size() < 3) throw util::ConfigError("--dump-wetbulb 需要输出路径");
        return dump_wetbulb(args[2]);
    }
    if (cmd == "--dump-props") {
        if (args.size() < 3) throw util::ConfigError("--dump-props 需要输出路径");
        return dump_props(args[2]);
    }
    throw util::ConfigError("未知命令: " + cmd);
}
```

```
}  // namespace

int main_impl(int argc, char** argv) {
    try {
        std::vector<std::string> args(argv, argv + argc);
        return real_main(args);
    } catch (const util::AssertError& e) {
        std::fprintf(stderr, "ERR assertion: %s\n", e.what());
        return 1;
    } catch (const util::ConfigError& e) {
        std::fprintf(stderr, "ERR config: %s\n", e.what());
        return 2;
    } catch (const std::exception& e) {
        std::fprintf(stderr, "ERR runtime: %s\n", e.what());
        return 1;
    }
}

#ifdef _WIN32
int wmain(int argc, wchar_t** argv) {
    try {
        std::vector<std::string> args;
        args.reserve(argc);
        // ⚠ 跳过 argv[0]：那是指向本 exe 的路径，本身就含中文
        //    （H:\2026 数学建模\cpp\build\cpp_sim.exe），不参与语义。
        args.emplace_back("cpp_sim");
        for (int i = 1; i < argc; ++i) args.push_back(narrow(argv[i]));
        return real_main(args);
    } catch (const util::AssertError& e) {
        std::fprintf(stderr, "ERR assertion: %s\n", e.what());
        return 1;
    } catch (const util::ConfigError& e) {
        std::fprintf(stderr, "ERR config: %s\n", e.what());
        return 2;
    } catch (const std::exception& e) {
        std::fprintf(stderr, "ERR runtime: %s\n", e.what());
        return 1;
    }
}
#else
int main(int argc, char** argv) { return main_impl(argc, argv); }
#endif

// ======== 文件：include/config.hpp ========
// -*- coding: utf-8 -*-
// config.hpp —— 全局常量。
//
// 本文件是各功能模块共用常量的**唯一来源**。
// 各模块专属的参数（物性、边界、时长）不放这里，由 params.txt 传入。
//
```

```
#pragma once

#include <array>
#include <string>

namespace cfg {

// ---------------------------------------------------------------------------
// 几何与初始条件（任务给定，各模块共用）          全局配置
// ---------------------------------------------------------------------------
inline constexpr double R_CM = 2.0;
inline constexpr double R_M = R_CM * 1e-2;      // 0.02 m
inline constexpr double L_CM = 25.0;
inline constexpr double T0_C = 28.0;
inline constexpr double T0_K = T0_C + 273.15;   // 301.15
inline constexpr double C0 = 2.55;              // kg/kg（干基）

// ---------------------------------------------------------------------------
// 蒸发（相变）项 —— 各模块统一的量纲与公式
// ---------------------------------------------------------------------------
// λ(T_s) = LATENT_A − LATENT_B·T_s[℃]，自变量钳位在 LATENT_T_RANGE 内
inline constexpr double LATENT_A = 2.501e6;     // J/kg，λ(0 ℃)
inline constexpr double LATENT_B = 2.36e3;      // J/(kg·K)
inline constexpr double LATENT_T_LO = 0.0;      // ℃
inline constexpr double LATENT_T_HI = 50.0;     // ℃

// 相变量纲开关：True = 按上式计入表面能量平衡（默认开启）。
// 代码里**只有这一个开关**：关闭它（λ=0）即退化为"传热传质解耦"的对照口径。
inline constexpr bool LATENT_ON = true;         //
// ---------------------------------------------------------------------------
// 表面对流换热的辐射补充（按 20% 折算）
// ---------------------------------------------------------------------------
// ⚠ H_RAD 在**各模块的配置文件** 里加上去（H_CONV = 25.0 + cfg.H_RAD = 31.72），
//   并不由本文件提供 H_CONV。
inline constexpr double H_RAD = 6.72;           // W/(m²·K)

// ---------------------------------------------------------------------------
// 相变项的两个旧闭包 —— PHASE_MODEL='film' 下**两者都被取代**，默认全关
//
// ---------------------------------------------------------------------------
// Φ（对标露点，EXT_SELFLIM）与 ϕ（对标湿球，EVAP_CAP_WETBULB）是同一约束的
// 两种闭包，**不可同开**；显式液膜模型（PHASE_MODEL='film'）把两者都取代：
// 表面温度由边界条件直接给定，传质系数不再需要任何自限因子。
inline bool EXT_SELFLIM = false;        // 的互斥修正结果
inline bool EVAP_CAP_WETBULB = false;   //（film 模型取代）

// ---------------------------------------------------------------------------
// 表面「两阶段相变份额」模型 —— 液膜图像下的预热段 / 恒温段
//
```

```
// ------------------------------------------------------------------------
//   预热段（0 ≤ t < WETBULB_T_SWITCH）：Robin + 相变汇按 ε_preheat 折减
//   恒温段（t ≥ WETBULB_T_SWITCH）：液膜把 T_s 钉在湿球温度（罚函数 Dirichlet），
//                                   物料内无相变源（ε_hold = 0）
inline constexpr double PHASE_FRACTION_PREHEAT = 0.03;   // ε（预热段）
inline constexpr double PHASE_FRACTION_HOLD = 0.0;       // ε（恒温段）
inline constexpr bool   WETBULB_SURFACE = true;
inline constexpr double WETBULB_T_SWITCH = 14400.0;      // s（= 工况曲线 覆盖终点 4
h）

// 液膜存在性判据（**默认开启**）。恒温段内只在"液膜确实存在"时才钉 T_w：
//   ratio = ṁ''_需 / ṁ''_max，w = clip((ratio − (1−band)) / (2·band), 0, 1)
// ⚠ 必须用**连续权重**且带宽要足够大：硬判据会把"含/不含相变"两解的微小差异
//   放大成 7.5 K 的温度差（实测），直接冲垮 10% 判据。band=1.0 时
//   w = min[1, ratio/2]，液膜"完整"要求 ratio ≥ 2。
inline constexpr bool   WETBULB_GATE = true;
inline constexpr double WETBULB_GATE_BAND = 1.00;        //
// 液膜形成过渡 [s]：0 = 硬切换（t_sw 处 T_s 会瞬间跳降 ≈ 8–9 K，属数值假象）
inline constexpr double WETBULB_RAMP = 600.0;
inline constexpr double H_PENALTY = 1.0e5;               // 罚函数 Robin，实现
Dirichlet

// ------------------------------------------------------------------------
// 数值通用设置
// ------------------------------------------------------------------------
inline constexpr double PICARD_TOL = 1e-12;
inline constexpr int    PICARD_MAX = 40;
inline constexpr double C_FLOOR = 0.0;

// ------------------------------------------------------------------------
// 输出网格（按工艺要求设定）全局配置
// ------------------------------------------------------------------------
// 每 0.1 cm 一个输出位置；NR 取 20 的整数倍时节点恰好落在这张网格上
// ⇒ 输出**零插值误差**。
inline constexpr int N_FULL = 21;                        // 0.0 … 2.0 cm
inline constexpr int N_FULL_SHRINK = 20;                     // 0.0 … 1.9 cm
inline constexpr std::array<double, 5> R_TABLE_CM{{0.0, 0.5, 1.0, 1.5, 2.0}};
inline constexpr std::array<double, 4> R_TABLE_CM_SHRINK{{0.0, 0.5, 1.0, 1.5}};

// 输出精度：所有结果保留四位小数
inline constexpr int EXCEL_ROUND = 4;

// ------------------------------------------------------------------------
// 时间步钟的归位网格
// ------------------------------------------------------------------------
// 长时程（10⁵–10⁶ 步）下 `t += dt` 会累积浮点漂移，导致"整秒/整分/整时记录"
// 这类精确匹配漏记。所有 dt 都是 0.1 s 的整数倍，故每步把 t 归位到最接近的
// 1e-9 网格上；对十分秒再做一次精确归位。
```

```
inline constexpr double TIME_SNAP_GRID = 1e-9;

// 单位换算
inline constexpr double KELVIN = 273.15;

}  // namespace cfg

// ======== 文件：include/util.hpp ========
// -*- coding: utf-8 -*-
// util.hpp —— 时间归位、数值格式化、断言式报错。
#pragma once

#include <cmath>
#include <cstdio>
#include <stdexcept>
#include <string>

#include "config.hpp"

namespace util {

// ---------------------------------------------------------------------------
// 时间归位                          对应 移动边界求解模块:331
// ---------------------------------------------------------------------------
// Python:  t = round(t + dt, 9)
// 用 nearbyint（当前舍入模式 = FE_TONEAREST，即"四舍六入五成双"）复刻 Python
// round() 的平局语义。所有 dt 都是 0.1 s 的整数倍 ⇒ 归位结果恰好落在十分秒上，
// `abs(t/60 - round(t/60)) < 1e-9` 这类记录判据才能精确命中。
inline double round9(double x) {
    return std::nearbyint(x * 1e9) / 1e9;
}

// ---------------------------------------------------------------------------
// 数值格式化 —— 与 numpy 的 repr 等价（17 位有效数字，可往返）
// ---------------------------------------------------------------------------
inline std::string repr(double v) {
    char buf[40];
    std::snprintf(buf, sizeof buf, "%.17g", v);
    return std::string(buf);
}

inline std::string fmt(double v, int dp = 4) {
    char buf[64];
    std::snprintf(buf, sizeof buf, "%.*f", dp, v);
    return std::string(buf);
}

// ---------------------------------------------------------------------------
// 错误 —— 对应断言失败与配置错误两类。
// 打印到 stderr 并让 main 返回 2（配置/断言错误），与 1（求解失败）区分开。
```

```
// ------------------------------------------------------------------------
class ConfigError : public std::runtime_error {
public:
    explicit ConfigError(const std::string& what)
        : std::runtime_error(what) {}
};

class AssertError : public std::runtime_error {
public:
    explicit AssertError(const std::string& what)
        : std::runtime_error(what) {}
};

// 对应 Python 的 assert：**不因 -DNDEBUG 失效**（本工程刻意不定义 NDEBUG，
//   因为网格、ξ 网格与记录器的自洽性都依赖活断言）。
inline void require(bool cond, const std::string& msg) {
    if (!cond) throw AssertError(msg);
}

}  // namespace util

// ======== 文件：include/params.hpp ========
// -*- coding: utf-8 -*-
// params.hpp —— Python ⇔ exe 的数据交换：key=value 参数文件、CSV 数值表、CSV 写出。
//
// 【为什么用 ASCII 相对路径】
//   `H:\2026 数学建模\` 含中文。MinGW 的窄字符 argv 按 ANSI 码页（936）解码，
//   中文路径进 argv 会乱码。对策是**双保险**：
//     · Python 把子进程 cwd 设成 cpp\ 项目根，只传纯 ASCII 的相对路径；
//     · exe 用 `-municode` 编译，wmain 拿宽字符命令行；params 文件内部的
//       路径也都是 ASCII 相对路径，fopen 时用 std::filesystem::path 走宽 API。
//   这样中文只出现在"进程当前目录"这一个由 CreateProcessW 正确设置的地方。
#pragma once

#include <fstream>
#include <map>
#include <string>
#include <vector>

namespace io {

// ------------------------------------------------------------------------
// 路径：窄（ASCII）→ 宽，并打开文件
// ------------------------------------------------------------------------
// 只接受 ASCII；非 ASCII 会被显式拒绝（而不是静默乱码）。
std::wstring widen_ascii(const std::string& s);
std::ifstream ifstream_w(const std::string& ascii_path);
std::ofstream ofstream_w(const std::string& ascii_path),
              ofstream_w_binary(const std::string& ascii_path);
```

```
void ensure_parent_dir(const std::string& ascii_path);

// ---------------------------------------------------------------------------
// key=value 参数文件
// ---------------------------------------------------------------------------
class Params {
public:
    // 逐行 `<key> = <value>`；`#` 起注释；空行忽略。键与值两端去空白。
    void load(const std::string& path);
    void set(const std::string& k, const std::string& v);

    bool has(const std::string& k) const;
    std::string str(const std::string& k, const std::string& def = "") const;

    double num(const std::string& k, double def) const;
    double num_req(const std::string& k) const;   // 缺失即 ConfigError
    int    inum(const std::string& k, int def) const;
    bool   flag(const std::string& k, bool def) const;  // 1/true/yes/on → true

    const std::map<std::string, std::string>& all() const { return kv_; }

private:
    std::map<std::string, std::string> kv_;
};

// ---------------------------------------------------------------------------
// CSV 数值表（第 0 列是自变量）
// ---------------------------------------------------------------------------
class Table {
public:
    // 读 CSV：跳过 `#` 注释行；第一个非注释行若是非数值表头则丢弃。
    void load(const std::string& path);

    size_t nrows() const { return x_.size(); }
    size_t ncols() const { return cols_.empty() ? 0 : cols_.size(); }
    double x(size_t i) const { return x_[i]; }
    double at(size_t col, size_t i) const { return cols_[col][i]; }
    const std::vector<double>& col(size_t c) const { return cols_[c]; }
    const std::vector<double>& xvec() const { return x_; }

    // 严格递增检查（单调表才可安全插值）
    bool monotone_x() const;

private:
    std::vector<double> x_;
    std::vector<std::vector<double>> cols_;
};

// ---------------------------------------------------------------------------
// CSV 写出（流式，`%.17g`；需要 4 位小数时传 dp=4）
```

```
// ----------------------------------------------------------------------------
class CsvWriter {
public:
    CsvWriter() = default;
    CsvWriter(const std::string& path, const std::string& header);
    void open(const std::string& path, const std::string& header);
    void row(const double* v, size_t n, int dp = 17);
    void row(const std::vector<double>& v, int dp = 17);
    void close();
    bool is_open() const { return f_.is_open(); }
    ~CsvWriter();

    CsvWriter(const CsvWriter&) = delete;
    CsvWriter& operator=(const CsvWriter&) = delete;

private:
    std::ofstream f_;
    std::vector<char> buf_;
};

}  // namespace io

// ======== 文件：include/grid.hpp ========
// -*- coding: utf-8 -*-
// grid.hpp —— 节点中心有限体积网格 + 界面系数。
//
// 节点 i = 0..NR 位于 r_i = i·dr，dr = R/NR。
// 只要 NR 是 20 的整数倍，节点就恰好落在 0, 0.1, …, 2.0 cm —— 与输出网格
// 逐点重合，输出**零插值误差**。
//
// 控制体（单位长度）：
//     cell 0  : [0, dr/2]             V = π (dr/2)²
//     cell i  : [r_i−dr/2, r_i+dr/2]  V = 2π i dr²
//     cell NR : [R−dr/2, R]           V = π dr² (NR − 1/4)
// 内面（节点 i 与 i+1 之间）面积 A_i = 2π(i+½)dr；外表面 A_R = 2πR。
//
// 该半格中心格式对 r→0 附近的抛物型剖面精确，整体空间二阶。
#pragma once

#include <vector>

namespace mesh {

struct Grid {
    int NR = 0;
    double dr = 0.0;
    double R = 0.0;
    std::vector<double> r;    // 节点位置，长度 NR+1
    std::vector<double> V;    // 控制体体积（单位长度），长度 NR+1
    std::vector<double> Af;   // 内部面面积，长度 NR
```

```
    double AR = 0.0;          // 外表面面积 2πR
};

Grid build_grid(int NR, double R_m);

// 几何自洽：ΣV_i = πR²。返回相对误差；超限则抛 AssertError。
// 相对误差超过 1e-12 即抛断言错误
double check_grid(const Grid& g);

// 把到中心的距离 (cm) 映射到网格节点下标；要求恰好落在节点上。
// 偏差超过 1e-9 cm 时报错。
std::vector<int> node_index(const std::vector<double>& r_cm, const Grid& g);

// ---------------------------------------------------------------------------
// 界面系数的插值方式                      对应 physics.FACE_KIND
// ---------------------------------------------------------------------------
//   Harmonic     —— 2ab/(a+b)。**当 a、b 是"两侧各自常数的 D"时才精确**
//                   （界面处有跳跃）；对光滑变化的 D 只有一阶精度，
//                   且系统性**低估**通量。
//   Arithmetic   —— (a+b)/2。对**光滑变化**的 D 是面中点值的二阶近似。
// 本问题的 D(C) 连续光滑但跨两个数量级，默认用**算术平均**；Harmonic 保留作对照。
enum class FaceKind { Arithmetic, Harmonic };

inline double harmonic(double a, double b) {
    const double s = a + b;
    return s > 0.0 ? 2.0 * a * b / s : 0.0;
}

inline double face_avg(double a, double b, FaceKind kind = FaceKind::Arithmetic) {
    if (kind == FaceKind::Arithmetic) return 0.5 * (a + b);
    return harmonic(a, b);
}

// 面系数 α_i = A_i · face_val_i / dr，长度 NR（i = 0..NR-1）
inline void face_alpha_from(const std::vector<double>& face_val, const Grid& g,
                            std::vector<double>& out) {
    out.resize(g.Af.size());
    for (std::size_t i = 0; i < g.Af.size(); ++i) {
        out[i] = g.Af[i] * face_val[i] / g.dr;
    }
}

}  // namespace mesh

// ======== 文件：include/props.hpp ========
// -*- coding: utf-8 -*-
// props.hpp —— 内置三套物性方案 + 蒸发（相变）项。
//
```

```
//      方案 A（常数物性）
//          ρ = 820、cp = 2600、k = 0.36；D = 7e-9·exp(-0.89/C)
//      方案 B（质量分数混合律）
//          ρ = 650 + 128C；cp = 1450 + 2736·C/(1+C)；k = 0.21 + 0.38·C/(1+C)
//          D = 2.4e-3·exp(-0.45/C)·exp(-3850/T)
//      方案 C（低导热变物性）
//          ρ = 760 + 90C；cp = 1850 + 2150·C/(1+C)；k = 0.12 + 0.20·C/(1+C)
//          D = 4.2e-4·exp(-0.30/C)·exp(-3850/T)
//
// ⚠ 三套 D 的指数都是**分数线读法**（−b/C）。旧读法 exp(−b·C) 下 D 随失水而
//   **增大**，是"越干越好失水"，与降速干燥的物理相反，烘干时长会被严重低估。
#pragma once

#include <memory>
#include <string>
#include <vector>

#include "grid.hpp"

namespace phys {

using mesh::FaceKind;
using mesh::Grid;

// ============================================================================
// 一套物性：ρ(C)、cp(C)、k(C)、D(C,T_K)，以及沿节点求面系数的助手。
// ============================================================================
class Props {
public:
    virtual ~Props() = default;

    virtual double rho(double C) const = 0;
    virtual double cp(double C) const = 0;
    virtual double k(double C) const = 0;
    virtual double D(double C, double T_K) const = 0;
    virtual std::string name() const = 0;

    // ---- 面系数（先取面值、再经 face_alpha_from 转系数）----
    // 这三个是**具体实现**，靠虚函数 rho/cp/k/D 取面值 ⇒ 子类（如 BlendProps）
    // 只要重写那四个虚函数，面系数自动跟着混。
    void alpha_D(const std::vector<double>& C, const std::vector<double>& T,
                 const Grid& g, std::vector<double>& out,
                 FaceKind kind = FaceKind::Arithmetic) const;
    void alpha_k(const std::vector<double>& C, const Grid& g,
                 std::vector<double>& out,
                 FaceKind kind = FaceKind::Arithmetic) const;
    void cap_heat(const std::vector<double>& C, const Grid& g,
                  std::vector<double>& out) const;

    // 物性是否与 C 无关（决定是否需要 Picard）。
```

```
    // 虚函数：BlendProps 恒返回 false（混合物性与 C 有关）。
    virtual bool is_constant() const;
};

using PropsPtr = std::shared_ptr<Props>;

// D_formula: 'exp_neg_inv_C'（默认读法）| 'legacy_exp_neg_C'（旧读法，仅供对照）
PropsPtr props_a(const std::string& D_formula = "exp_neg_inv_C",
                 double D_prefactor = 7.0e-9, double D_const = 0.89);
PropsPtr props_b();
PropsPtr props_c();

// 按名字取（params.txt 里的 props_pre / props_hold 用）
PropsPtr props_by_name(const std::string& name);

// ============================================================================
// 蒸发（相变）项 —— 全程序**唯一**的相变项实现
// ============================================================================
// 求解器、移动边界求解与守恒校核三处都调用下面这几个函数，**不能各写一份**。
// 否则"求解"与"校核"用的不是同一个离散式，守恒恒等式会残差爆掉。

// 表面干基密度 ρ_d = ρ(C_s)/(1+C_s)  [kg/m³]
inline double surface_dry_rho(const Props& p, double C_s) {
    return p.rho(C_s) / (1.0 + C_s);
}

// 蒸发质量通量 ṁ'' = ρ_d·k_m·(C_s − C_∞)  [kg/(m²·s)]
// 量纲：kg/m³ · m/s · (kg/kg) = kg/(m²·s) ✓
inline double evap_mass_flux(const Props& p, double k_m, double C_s, double C_inf)
{
    return surface_dry_rho(p, C_s) * k_m * (C_s - C_inf);
}

// 水的汽化潜热 λ(T) = LATENT_A − LATENT_B·T[℃]  [J/kg]（0–50 ℃ 线性化）。
// ⚠ 自变量**先钳位**再代入线性式，避免外推。
double latent_heat(double T_C);

// 表面相变吸热功率（单位长度圆柱，W/m）：q = ε·A_R·λ(T_s)·ṁ''
//
// fraction = ε —— **相变份额**：到达表面的水分中只有 ε 这一部分真正汽化，
//   其余 (1−ε) 以液态水形式被气流带离，不吸潜热。
//     ε = 1     到达即全部蒸发（对照口径）
//     ε = 0.03  预热段（默认）
//     ε = 0     恒温段（相变发生在物料外的液膜自由面上）
//
// h_fg 非 NaN 时用它覆盖 λ(T)；主结果一律走 λ(T_s)。
// AR 为标量（固定域 2πR）或逐层取值（移动边界 2πR(t)）由调用方决定。
//
// ⚠ `fraction` **先乘**：ε=0 必须给精确的 0.0，
```

```
//   不能出现 0.0 × Inf。
inline double evap_heat_per_length(const Props& p, double AR, double k_m,
                                   double C_s, double C_inf, double T_s_C,
                                   double h_fg, double fraction) {
    const double lam = (h_fg == h_fg) ? h_fg : latent_heat(T_s_C);  // NaN ⇒ λ(T_s)
    return fraction * lam * AR * evap_mass_flux(p, k_m, C_s, C_inf);
}

}  // namespace phys

// ======== 文件：include/psychro.hpp ========
// -*- coding: utf-8 -*-
// psychro.hpp —— 湿空气热力学 + 两阶段液膜表面模型。
//
//
// 【核心物理】干燥理论的标准图像：**恒速段表面覆盖连续液膜，汽化相当于自由
//   液面蒸发**，传入热量全部用于汽化 ⇒ 表面温度 = 空气湿球温度 Tw。
//   表面一旦变干进入**降速段**，没有液膜、T_s 回升向 T∞。
//
// 【两阶段】
//   预热段（t < t_switch）：Robin + 相变汇按 ε_preheat 折减
//   恒温段（t ≥ t_switch）：液膜把表面钉在 Tw（罚函数 Dirichlet），物料内无相变源
#pragma once

#include <cmath>
#include <cstddef>
#include <functional>
#include <limits>
#include <map>
#include <utility>

namespace psychro {

// ---------------------------------------------------------------------------
// 常数
// ---------------------------------------------------------------------------
inline constexpr double P_ATM = 101325.0;   // Pa
inline constexpr double M_W = 0.018;        // kg/mol，水
inline constexpr double M_DA = 0.02896;     // kg/mol，干空气
inline constexpr double R_G = 8.314;        // J/(mol·K)
inline constexpr double EPS = M_W / M_DA;   // ≈ 0.622
inline constexpr double CP_AIR = 1005.0;    // J/(kg·K)，干空气定压比热

// 饱和水蒸气分压 [Pa]。Magnus 式，-45…60 ℃ 内优于 0.4%。
inline double p_sat(double T_C) {
    return 610.94 * std::exp(17.625 * T_C / (T_C + 243.04));
}
```

```
// 饱和水蒸气质量浓度 [kg/m³]（理想气体）
inline double rho_v_sat(double T_C) {
    return p_sat(T_C) * M_W / (R_G * (T_C + 273.15));
}

// 湿度比 W [kg/kg 干空气] → 水蒸气分压 [Pa]
inline double pv_of_W(double W) { return W * P_ATM / (EPS + W); }

// 湿度比 W → 水蒸气质量浓度 [kg/m³]（温度为空气温度）
inline double rho_v_of_W(double W, double T_C) {
    return pv_of_W(W) * M_W / (R_G * (T_C + 273.15));
}

// 露点温度 [℃]（由 p_sat(T_dp) = p_v,∞ 反解）
inline double dew_point(double W) {
    const double pv = pv_of_W(W);
    if (pv <= 0.0) return -std::numeric_limits<double>::infinity();
    const double ln = std::log(pv / 610.94);
    return 243.04 * ln / (17.625 - ln);
}

// ---------------------------------------------------------------------------
// 湿球温度
// ---------------------------------------------------------------------------
// 由液膜自由面的能量平衡定义：
//     h(T∞ − Tw) = λ(Tw)·k_m,air·[ρ_v,sat(Tw) − ρ_v,∞]
// 其中 k_m,air 由传热传质类比（Chilton–Colburn，Le^(2/3)≈1）给出
//     k_m,air = h / (ρ_air·c_p,air)
//
// 区间取 [−20, T∞]；两端同号（空气已饱和 ⇒ 无蒸发冷却）时**返回 T∞**
// —— 这是 scipy brentq 抛 ValueError 后 psychro 的显式回退，必须复刻。
double wet_bulb(double T_air_C, double C_inf, double h = 25.0,
                double cp_air = CP_AIR);

// 相变自限因子 Φ(T_s) ∈ [floor, ∞)（对标**露点**）。
//   T_s = T∞  ⇒ Φ = 1（= 标定原值，标定不变）
//   T_s = T_dp ⇒ Φ = 0（蒸发停止，不允许再冷）
// ⚠ Φ 与 ϕ 是同一约束的两种闭包；液膜模型下**两者都关闭**。
double selflim_factor(double T_s_C, double T_inf_C, double C_inf, double floor =
0.0);

// 「供热上限」因子 ϕ ∈ (0, 1]（对标**湿球**）：ϕ = min[1, ṁ''_max / ṁ''_需求]
double heat_cap_factor(double T_inf_C, double C_inf, double C_s, double k_m,
                       double rho_d, double h = 25.0);

// ---------------------------------------------------------------------------
// 液膜权重
// ---------------------------------------------------------------------------
// w ∈ [0,1]（**连续**，避免硬判据把微小差异放大成跳变）。
```

```
//   ratio = ṁ''_需 / ṁ''_max（供水能力 / 供热允许的蒸发上限）
//   ratio ≥ 1+band ⇒ w = 1（液膜完整，表面钉在 Tw）
//   ratio ≤ 1−band ⇒ w = 0（液膜消失，退回 Robin）
// 物理读法：w 就是 Schlünder 的**湿润表面分数 f**。
double film_weight(double ratio, double band);

// ---------------------------------------------------------------------------
// 两阶段表面模型
// ---------------------------------------------------------------------------
struct SurfVals {
    double h_eff = 0.0;    // 表面行的 Robin 系数
    double T_bc_K = 0.0;   // 表面行 Robin 驱动温度
    double eps = 1.0;      // 该时刻的相变份额 ε
};

class Surface {
public:
    // k_m / rho_d_fn 只在 gate=true（按"液膜是否还存在"自动判段）时必需。
    // rho_d_fn 为空 ⇒ 视为未提供（与 Python 传 None 同义）。
    Surface(double t_switch, double eps_preheat, double eps_hold, bool wetbulb,
            double h_conv, double h_pen, bool gate, double t_ramp,
            double gate_band, double k_m,
            std::function<double(double)> rho_d_fn);

    // surf(t, T_inf_K, C_inf, C_s) -> (h_eff, T_bc_K, ε)
    SurfVals operator()(double t, double T_inf_K, double C_inf, double C_s) const;

    // 便于自检/出图直接取湿球温度
    double Tw(double T_inf_C, double C_inf) const;

    bool wetbulb() const { return wetbulb_; }
    double t_switch() const { return t_switch_; }

private:
    double t_switch_, eps_preheat_, eps_hold_;
    bool wetbulb_;
    double h_conv_, h_pen_;
    bool gate_, has_rho_d_;
    double t_ramp_, gate_band_, k_m_;
    std::function<double(double)> rho_d_fn_;

    // 湿球求根缓存：长时程下恒温段 Tw 是常数，避免 10⁵–10⁶ 步反复求根。
    // 键 = (round(T_inf,6), round(C_inf,9))；超 4096 条清空（与 同）。
    mutable std::map<std::pair<double, double>, double> tw_cache_;
};

}  // namespace psychro

// ======== 文件：include/transition.hpp ========
```

```
// -*- coding: utf-8 -*-
// transition.hpp —— 预热段 → 恒温段的**连续过渡**（σ 混合）。
//
//
// 【问题】任务要求只给了工况曲线的 0–4 h 实测边界曲线，t > 4 h 留白 ⇒ 显式假设恒温
//   (T∞=50 ℃、C∞=0.0499)。若在 t_sw = 4 h 处**硬跳变**，边界曲线与**物性**都会
//   留下人为间断：ρ 在 4 h 处跳了约 170 kg/m³（21%），比边界的跳变大三个量级。
//
// 【做法】两条都用**同一个** σ(t)（紧支撑 C¹ smoothstep）连续混合：
//     T∞(t) = (1−σ)·m(t) + σ·T_hold,        C∞(t) 同理
//     P(t)  = (1−σ)·P_预热 + σ·P_恒温,        P = ρ, c_p, k, D
//   σ 在 [t_sw, t_sw+w] 上由 0 升到 1。
//
// ⚠ 两条通道的窗口**故意不同**（见 全周期模块的过渡窗口设置）：
//     物性 [1.5, 4] h —— 温度在 1.35–2.11 h 稳定，到 4 h 物性已切完，正好接恒温段
//     边界 [4, 4+w] h —— 工况曲线的保形插值 **不能外推**，GPR 只能在实测段之外接管
//   所以"物性的过渡在实测段内完成、边界的过渡在实测段之外发生"。
//
// ⚠ **GPR 用离线表代替在线拟合**：高斯过程回归的完整实现不适合放进求解器，
//   故由数据准备脚本离线拟合一次（固定随机种子以保证可复现），
//   把后验均值 m(t) 求值到等距表，C++ 用单调 Hermite 插值求值。
#pragma once

#include <memory>
#include <string>

#include "interp.hpp"
#include "props.hpp"

namespace trans {

// 紧支撑 C¹ smoothstep：t_c ± width_s/2 内由 0 升到 1。
// w=0 时把 x 钳到 {0,1} ⇒ 退化为**阶跃**（与硬切换逐位一致）。
inline double sigma(double t, double t_center, double width_s) {
    const double denom = (width_s > 1e-9) ? width_s : 1e-9;
    double x = (t - (t_center - width_s / 2.0)) / denom;
    if (x < 0.0) x = 0.0;
    if (x > 1.0) x = 1.0;
    return x * x * (3.0 - 2.0 * x);
}

// ---------------------------------------------------------------------------
// GPR 后验均值表（Python 预算）
// ---------------------------------------------------------------------------
class GprTable {
```

```
public:
    void load(const std::string& path);      // 表列：t_s, mT_C, mC_kg_kg
    double mT(double t) const { return pT_(t); }
    double mC(double t) const { return pC_(t); }
    bool empty() const { return pT_.empty(); }

private:
    interp::Pchip pT_, pC_;
};

// ---------------------------------------------------------------------------
// 两套物性按**同一个** σ(t) 连续混合
// ---------------------------------------------------------------------------
// ⚠ 为什么需要 set_t：主推进循环只把 t 传给 dt_schedule 与 bc，**不传给
//   props**。故由边界闭包 bc(t) 在每步开头调 set_t(t)，使 σ 进入求解器 ——
//   这样不必改动求解器。
//   生效的是 σ(t+Δt)（CN 的 n+1 半边），Δt=1 s 相对 ≥1.5 h 的过渡窗可忽略。
class BlendProps : public phys::Props {
public:
    BlendProps(phys::PropsPtr pre, phys::PropsPtr hold,
               double t_start_s, double t_end_s);

    void set_t(double t) { s_ = sigma(t, t_c_, w_); }
    double s() const { return s_; }
    double sigma_at(double t) const { return sigma(t, t_c_, w_); }

    double rho(double C) const override;
    double cp(double C) const override;
    double k(double C) const override;
    double D(double C, double T_K) const override;

    std::string name() const override { return name_; }
    bool is_constant() const override { return false; }   // 混合物性与 C 有关 ⇒ 必
须走 Picard

private:
    phys::PropsPtr pre_, hold_;
    double t_a_, t_b_, t_c_, w_;
    double s_ = 0.0;
    std::string name_;
};

// ---------------------------------------------------------------------------
// 两阶段边界驱动
// ---------------------------------------------------------------------------
enum class BcMethod { PchipOnly, PchipStep, PchipGp };

struct BcValues {
    double T_inf_K = 0.0;
    double C_inf = 0.0;
```

```
};

class BcSmooth {
public:
    // Tp/Cp 为工况曲线的保形插值（由调用方持有）；gpr 可为 nullptr（非 gp 模式）。
    // blend 非空时，每次求值会顺带 set_t(t) —— 这是 σ 进入**物性**的唯一通道。
    BcSmooth(const interp::Pchip* Tp, const interp::Pchip* Cp,
             const GprTable* gpr, double t_switch, double width_h,
             double T_hold, double C_hold, BcMethod method,
             BlendProps* blend = nullptr);

    BcValues operator()(double t) const;

    double t_switch() const { return t_sw_; }
    double width_h() const { return w_ / 3600.0; }

private:
    const interp::Pchip* Tp_;
    const interp::Pchip* Cp_;
    const GprTable* gpr_;
    double t_sw_, w_, t_c_;
    double T_hold_, C_hold_;
    BcMethod method_;
    BlendProps* blend_;
};

}  // namespace trans

// ======== 文件：include/interp.hpp ========
// -*- coding: utf-8 -*-
// interp.hpp —— 单调三次 Hermite 插值（Fritsch-Carlson），逐行复刻
//               scipy.interpolate.PchipInterpolator。
//
// 两条用途：
//   1. 工况曲线的 T_inf(t) / C_inf(t) 控制曲线（0 ≤ t ≤ 14400 s）；
//   2. Python 预算好的 GPR 后验均值表 m(t) 的求值（过渡窗内）。
//
// 【为什么必须逐行复刻】
//   插值结果会逐时间步进入边界条件，任何系统性偏差都会污染整条轨迹。
//   复刻要点（scipy `PchipInterpolator._find_derivatives` / `_edge_case`）：
//     · 内部节点：带符号检验的加权调和平均
//         d_k = 0                                  若 m_{k-1}、m_k 异号或任一为 0
//         d_k = (w1 + w2) / (w1/m_{k-1} + w2/m_k)   否则
//       其中 w1 = 2h_k + h_{k-1}、w2 = h_k + 2h_{k-1}
//     · 端点：单侧三点估计 + 保形修正
//         d = ((2h0 + h1)·m0 − h0·m1) / (h0 + h1)
//         sign(d) != sign(m0)                          ⇒ d = 0
//         sign(m0) != sign(m1) 且 |d| > 3|m0|          ⇒ d = 3·m0
```

```
//     · 求值：三次 Hermite 基（与 scipy 的 Horner 写法代数等价）
//
// ⚠ 真实路径上 PCHIP **从不外推** —— make_bc_smooth 只在 t ≤ t_sw 时调它。
//   本实现仍对区间外做端点钳位（`extrapolate=False` 语义），以免误用。
#pragma once

#include <cstddef>
#include <vector>

namespace interp {

class Pchip {
public:
    Pchip() = default;
    Pchip(std::vector<double> x, std::vector<double> y) { build(std::move(x),
std::move(y)); }

    // x 必须严格递增。自动按 Fritsch-Carlson 求各节点导数。
    void build(std::vector<double> x, std::vector<double> y);

    double operator()(double t) const;
    double slope(double t) const;          // 解析导数（分段三次的导数）

    // 定位 t 所在区间的下标（落在区间外时钳到首/末段）
    std::size_t segment(double t) const;

    bool empty() const { return x_.empty(); }
    std::size_t size() const { return x_.size(); }
    double x_front() const { return x_.front(); }
    double x_back() const { return x_.back(); }

private:
    std::vector<double> x_, y_, d_;        // 节点、值、节点导数
};

// 等距表上的单调 Hermite 插值（GPR 表用；内部直接建 Pchip）
class Table1D {
public:
    Table1D() = default;
    Table1D(std::vector<double> x, std::vector<double> y) : p_(std::move(x),
std::move(y)) {}
    void build(std::vector<double> x, std::vector<double> y)
{ p_.build(std::move(x), std::move(y)); }
    double operator()(double t) const { return p_(t); }
    bool empty() const { return p_.empty(); }

private:
    Pchip p_;
};
```

```
}  // namespace interp

// ======== 文件：include/brent.hpp ========
// -*- coding: utf-8 -*-
// brent.hpp ——  Brent 求根，逐行复刻 scipy.optimize.brentq。
//
// 唯一用途：psychro.wet_bulb 求湿球温度 Tw，即解
//     h(T∞ − Tw) = λ(Tw)·k_m,air·[ρ_v,sat(Tw) − ρ_v,∞]
//
// 【为什么必须复刻算法而不是"随便求个根"】
//   Tw 通过罚函数 Robin（h_pen = 1e5）把表面温度钉住：T_s − Tw ~ 3e-4 × 驱动温差
//   ⇒ Tw 上 1e-11 K 的误差仍在总门槛 1e-7 K 之下。用同款算法可保证 1e-12 K。
//   更关键的是**失败回退**：scipy 在两端点同号（空气已饱和）时抛 ValueError，
//   psychro 捕获后返回 T。这条路径是可达的，静默返回别的值会污染 Dirichlet 驱动。
#pragma once

#include <cmath>
#include <functional>
#include <stdexcept>

namespace root {

// scipy.optimize.brentq 的经典实现（scipy/optimize/Zeros/brentq.c）
//
// 返回 [xa, xb] 内的根。若两端同号则抛 std::domain_error（对应 scipy 的
// ValueError），由调用方决定回退策略。
inline double brentq(const std::function<double(double)>& f, double xa, double xb,
                     double xtol = 2e-12, double rtol = 8.881784197001252e-16,
                     int iter = 100) {
    double xpre = xa, xcur = xb;
    double xblk = 0.0, fblk = 0.0, spre = 0.0, scur = 0.0, sbis = 0.0, delta = 0.0;
    double stry, dpre, dblk;

    double fpre = f(xpre);
    double fcur = f(xcur);
    if (fpre == 0.0) return xpre;
    if (fcur == 0.0) return xcur;
    if ((fpre > 0.0) == (fcur > 0.0)) {
        throw std::domain_error("brentq: 两端点函数值同号，无法定界");
    }

    for (int i = 0; i < iter; ++i) {
        if (fpre != 0.0 && fcur != 0.0 && ((fpre > 0.0) != (fcur > 0.0))) {
            xblk = xpre;
            fblk = fpre;
            spre = scur = xcur - xpre;
```

```
        }
        if (std::fabs(fblk) < std::fabs(fcur)) {
            // 交换 (xpre,xcur,xblk)，保持 |fcur| ≤ |fblk|
            xpre = xcur; xcur = xblk; xblk = xpre;
            fpre = fcur; fcur = fblk; fblk = fpre;
        }

        delta = (xtol + rtol * std::fabs(xcur)) / 2.0;
        sbis = (xblk - xcur) / 2.0;
        if (fcur == 0.0 || std::fabs(sbis) < delta) {
            return xcur;
        }

        if ((std::fabs(spre) > delta) && (std::fabs(fcur) < std::fabs(fpre))) {
            if (xpre == xblk) {
                // 割线插值
                stry = -fcur * (xcur - xpre) / (fcur - fpre);
            } else {
                // 反二次外推
                dpre = (fpre - fcur) / (xpre - xcur);
                dblk = (fblk - fcur) / (xblk - xcur);
                stry = -fcur * (fblk * dblk - fpre * dpre)
                       / (dblk * dpre * (fblk - fpre));
            }
            if (2.0 * std::fabs(stry) <
                std::min(std::fabs(spre), 3.0 * std::fabs(sbis) - delta)) {
                spre = scur;
                scur = stry;          // 接受这一步
            } else {
                spre = sbis;
                scur = sbis;          // 退化为二分
            }
        } else {
            spre = sbis;
            scur = sbis;
        }

        xpre = xcur;
        fpre = fcur;
        if (std::fabs(scur) > delta) {
            xcur += scur;
        } else {
            xcur += (sbis > 0.0 ? delta : -delta);
        }
        fcur = f(xcur);
    }
    return xcur;
}

}  // namespace root
```

```
// ======== 文件：include/solver.hpp ========
// -*- coding: utf-8 -*-
// solver.hpp —— 通用变系数 FVM + Crank–Nicolson + Picard + 三对角求解。
//
//
// 守恒方程（单位长度圆柱）
//   能量：ρ(C)cp(C)·V_i dT_i/dt = α^k_{i-1}(T_{i-1}-T_i) - α^k_i (T_i-T_{i+1}) +
S_i
//   质量：V_i dC_i/dt           = α^D_{i-1}(C_{i-1}-C_i) - α^D_i (C_i-C_{i+1})
//
// 时间离散（CN）
//   -½α_{i-1}u_{i-1}^{n+1} + [cap_i/Δt + ½(α_{i-1}+α_i)]u_i^{n+1} - ½α_i
u_{i+1}^{n+1}
//       = cap_i/Δt·u_i^n + ½G_i(u^n) + (n+1 层边界驱动)
//
// ⚠ **隐式半边与显式半边的面系数必须分开传**（alpha_impl=α(u^{n+1})、
//   alpha_expl=α(u^n)）。若两边都用迭代值，显式项被线性化，时间精度从二阶
//   掉到一阶（实测 2.0 → 1.1）。
#pragma once

#include <cmath>
#include <functional>
#include <string>
#include <vector>

#include "config.hpp"
#include "grid.hpp"
#include "props.hpp"
#include "psychro.hpp"
#include "transition.hpp"

namespace slv {

using Vec = std::vector<double>;

// ============================================================================
// 三对角求解
// ============================================================================
// Thomas 追赶法（O(n)，对角占优时稳定）。
//   a : (n-1) 下对角，a[j] 为第 j+1 行的下元
//   b : (n)   主对角
//   c : (n-1) 上对角，c[j] 为第 j 行的上元
//
// ⚠ 替代 LAPACK dgtsv：_assemble 出来的矩阵**严格对角占优**、裕量就是 cap/Δt
//   （量级极大），无主元 Thomas 向后稳定、增长因子为 1；dgtsv 的部分主元在这
//   个矩阵上什么也不做，只多一次行交换 ⇒ 两者每次求解一致到舍入。
void thomas(const Vec& a, const Vec& b, const Vec& c, const Vec& d, Vec& x);
```

```
// ============================================================================
// 半步与组装
// ============================================================================
// G(u)：广延通量 + 表面对流 + 可选源项
void rhs(const Vec& u, const Vec& alpha, const mesh::Grid& g, double beta,
         double u_inf, const Vec* source, Vec& F);

// 组装三对角 A = cap/Δt·I − ½L
void assemble(const Vec& alpha_impl, const mesh::Grid& g, double beta, double dt,
              const Vec& cap, Vec& a, Vec& b, Vec& c);

// 表面等效传质系数 = k_m · Φ(自限) · ϕ(供热上限)
//
// ⚠ 求解器（含移动边界）与守恒校核**必须走这一个函数**，否则质量与能量记的
//   不是同一份水，守恒恒等式立刻失配。
//   PHASE_MODEL='film' 下 cfg::EXT_SELFLIM 与 cfg::EVAP_CAP_WETBULB 均为 false
//   ⇒ 本函数退化为恒等映射；保留结构以便对照口径。
double surface_k_m(double k_use, double T_s_K, double T_inf_K, double C_inf,
                   const phys::Props* props, double C_s, double h_conv);

// CN 推进一个时间步（线性，系数已冻结）
void cn_step(const Vec& u, double dt, const mesh::Grid& g,
             const Vec& alpha_impl, const Vec& alpha_expl, double beta,
             double u_inf_n, double u_inf_np1, const Vec* source_n,
             const Vec* source_np1, const Vec* cap, const Vec* cap_old,
             Vec& out, double beta_n);

// ============================================================================
// 耦合单步（质量 + 能量，Picard 外层迭代）
// ============================================================================
struct SurfPair {
    psychro::SurfVals n, one;      // t^n 与 t^{n+1} 两层的取值
    bool on = false;
};

struct StepResult {
    int iters = 0;
    double err = 0.0;
};

StepResult step_coupled(Vec& T, Vec& C, double dt, const mesh::Grid& g,
                        phys::Props& props, const trans::BcValues& bc_n,
                        const trans::BcValues& bc_1, double h_conv, double k_m,
                        bool latent, double h_fg, const SurfPair& surf,
                        double tol, int pmax);

// ============================================================================
// 主推进循环
// ============================================================================
```

```
struct MarchConfig {
    double t_end = 0.0;
    double t_start = 0.0;
    double h_conv = 25.0;
    double k_m = 8.0e-7;
    bool latent = cfg::LATENT_ON;
    double h_fg = std::nan("");     // NaN ⇒ λ 取 λ(T_s)
    double tol = cfg::PICARD_TOL;
    int pmax = cfg::PICARD_MAX;

    Vec T0, C0;                     // 空 ⇒ 用 cfg 初值
    const psychro::Surface* surf = nullptr;

    std::function<trans::BcValues(double)> bc;
    std::function<double(double)> dt_schedule;
    // record(t, T, C)
    std::function<void(double, const Vec&, const Vec&)> record;
    // stop(t, T, C) -> bool
    std::function<bool(double, const Vec&, const Vec&)> stop;
    // tracker(t, dt, T_old, C_old, T_new, C_new, iters)
    std::function<void(double, double, const Vec&, const Vec&, const Vec&,
                       const Vec&, int)> tracker;
    // progress(t, steps, T, C, iters)
    std::function<void(double, int, const Vec&, const Vec&, int)> progress;
};

struct MarchResult {
    double t = 0.0;
    Vec T, C;
    int steps = 0;
    bool stopped = false;
    double iters_mean = 0.0;
    int iters_max = 0;
};

MarchResult march(const mesh::Grid& g, phys::Props& props,
                  const MarchConfig& mc);

}  // namespace slv

// ======== 文件：include/moving.hpp ========
// -*- coding: utf-8 -*-
// moving.hpp —— 收缩工况模块的 ξ-材料坐标移动边界求解器。
//
// 坐标变换（Landau / 材料坐标）
// ------------------------------
// 令 ξ = r/R(t) ∈ [0,1]，U(ξ,t) = C(ξR(t), t)。链式法则给出
//     ∂C/∂t|_r = ∂U/∂t|_ξ − (ξṘ/R)·∂U/∂ξ
// 代入 ∂C/∂t = (1/r)∂_r(Dr∂_rC)，得 ξ-坐标方程
//     ∂U/∂t = (1/(R²ξ))·∂_ξ(ξD∂_ξU) + (ξṘ/R)·∂U/∂ξ
```

```
            print('    注：允许空格 —— result4 中 r > R(t) 的列已收缩出材料域，'
                  '留空是设计如此')
        ok &= good
    print('=' * 74)
    print('格式核对：' + ('全部通过 ✓' if ok else '存在失败项 ✗'))
    return 0 if ok else 1


if __name__ == '__main__':
    raise SystemExit(main(sys.argv[1:]))

// ======== 文件：python/cli.py ========
# -*- coding: utf-8 -*-
"""
cli.py —— 无界面入口。**GUI 与它共用同一条代码路径**（runner/convert/postproc）。

    python -m cpp.python.cli build                 # 构建 exe
    python -m cpp.python.cli selfcheck             # 第一层自检
    python -m cpp.python.cli prep [--check]        # 生成 tables/（第二层验证）
    python -m cpp.python.cli run    preheat fullcycle duration shrinkage
                                                   # 求解 + 组装 npz / xlsx / csv
    python -m cpp.python.cli verify preheat fullcycle duration shrinkage
                                                   # 与标准结果逐键对比
    python -m cpp.python.cli all                    # prep -> run -> verify
"""

import argparse
import subprocess
import sys
import time
from pathlib import Path

import numpy as np

# 直接 `python cli.py` 时 __package__ 为空，相对导入会失败；补上项目根按模块重跑。
if __name__ == '__main__' and not __package__:
    import runpy
    import sys
    from pathlib import Path as _P
    sys.path.insert(0, str(_P(__file__).resolve().parents[2]))
    runpy.run_module('cpp.python.cli', run_name='__main__')
    raise SystemExit(0)

from . import BUILD, CPP, DATA, EXE, RUNS, ensure_dirs
from . import convert, runner, specs
```

```
# ---------------------------------------------------------------------------
def cmd_build(args):
    ps = CPP / 'build.ps1'
    r = subprocess.run(['powershell', '-NoProfile', '-ExecutionPolicy', 'Bypass',
                        '-File', str(ps)],
                       cwd=str(CPP))
    return r.returncode


def cmd_selfcheck(args):
    r = subprocess.run([str(EXE), '--selfcheck'], cwd=str(CPP))
    return r.returncode


def cmd_prep(args):
    from . import prep_tables
    return prep_tables.main(['--check'] if args.check else [])


# ---------------------------------------------------------------------------
def _duration_base_h():
    """取时长预测模块的结果作为对照基准（小时）。"""
    for p in (RUNS / 'duration' / 'out.npz', DATA / 'duration_fields.npz'):
        try:
            return float(np.load(p, allow_pickle=True)['t_end']) / 3600.0
        except Exception:                       # noqa: BLE001
            continue
    return 57.2655


def effect_decomposition(verbose=True):
    """收缩工况的效应分解：

    ① 固定半径 + 方案 C ── 物性单独的影响
    ② 收缩半径 + 方案 B ── 几何单独的影响
    """
    vals = []
    for spec, tag in ((specs.spec_shrinkage_case_props(), '① 固定 R + 物性方案 C'),
                      (specs.spec_shrinkage_case_geom(), '② 收缩 R(t) + 物性方案 B')):
        t0 = time.time()
        rc, rd = runner.run_sync(spec, log=(lambda s: None))
        s = convert._read_summary(rd)
        vals.append(float(s.get('t_end', np.nan)) / 3600.0)
        if verbose:
            print(f'    {tag}: t_end = {vals[-1]:.4f} h  '
                  f'（{time.time()-t0:.1f} s，退出码 {rc}）')
    return vals
```

```
def run_one(module, verbose=True, excel=False, publish=False, log=print,
            scenarios=False, T_inf_hold=None, c_inf=None):
    """跑一个模块并组装产物。返回 (ok, run_dir, data)。

    ⚠ 结果永远先写到 `runs/<module>/out.npz`；只有 `publish=True` 才覆盖
      `data/<模块>_fields.npz`（标准结果，默认不动）。
    """
    from common import config as cfg
    spec = specs.SPECS[module](T_inf_hold=T_inf_hold, c_inf=c_inf)
    t0 = time.time()
    if verbose:
        print(f'\n{"="*72}\n{module}  求解中…\n{"="*72}')
    rc, run_dir = runner.run_sync(spec, log=log if verbose else (lambda s: None))
    if rc != 0:
        print(f'  {module} 求解失败（退出码 {rc}）')
        return False, run_dir, None
    if verbose:
        print(f'  求解完成，{time.time()-t0:.1f} s')

    kw = {}
    if module == 'shrinkage':
        if verbose:
            print('  效应分解对照：')
        vals = effect_decomposition(verbose=verbose)
        kw = dict(cmp_labels=['① 固定半径 + 物性方案 C（分离物性效应）',
                              '② 收缩半径 + 物性方案 B（分离几何效应）',
                              '③ 收缩 + 物性方案 C'],
                  cmp_vals=vals + [None], q3_base_h=_duration_base_h())

    out_npz = Path(run_dir) / 'out.npz'
    d = convert.build(module, run_dir, out_npz=out_npz, excel=excel, **kw)
    if module == 'shrinkage' and kw:
        # ③ 就是本模块的主结果（回填后重写一次）
        d['cmp_vals'][2] = float(d['t_end']) / 3600.0
        np.savez(out_npz, **d)
    if publish:
        dst = cfg.DIR_DATA / f'{module}_fields.npz'
        np.savez(dst, **d)
        if verbose:
            print(f'  [publish] 已更新 {dst}')
    if verbose:
        _report(module, d)
    return True, run_dir, d


def _report(module, d):
    if module == 'preheat':
```

```
        print(f'  t_end = {d["t"][-1]:.0f} s   表面 T = {d["T"][-1][-1]-
273.15:.4f} ℃'
              f'   中心 C = {d["C"][-1][0]:.4f}   表面 C = {d["C"][-1][-1]:.4f}')
    elif module == 'fullcycle':
        print(f'  t_end = {d["t_end"]/3600:.2f} h   中心 C = {d["C"][-1][0]:.4f}'
              f'   质量失衡 {d["mass_rel"]:.2e}   能量失衡 {d["energy_rel"]:.2e}')
    else:
        print(f'  ★ t_end = {d["t_end"]/3600:.4f} h = {d["t_end"]/86400:.4f} 天'
              f'   （穿越 {d["t_cross"]/3600:.4f} h，确认 {d["t_conf"]/3600:.4f} h）')
    if module == 'shrinkage' and 'cmp_vals' in d:
        for lab, v in zip(d['cmp_labels'], d['cmp_vals']):
            print(f'    {lab}: {v:.4f} h')


def cmd_run(args):
    ensure_dirs()
    ok = True
    for module in (args.q or ['preheat']):
        o, rd, d = run_one(module, excel=args.excel, publish=args.publish)
        ok &= o
    return 0 if ok else 1


def cmd_verify(args):
    ok = True
    for module in (args.q or ['preheat']):
        ok &= convert_verify(module)
    return 0 if ok else 1


def convert_verify(module):
    """与参照 npz 对比。优先用 C++ 自己跑出来的 runs/<module>/out.npz。"""
    from . import verify as V
    out = RUNS / module / 'out.npz'
    if out.exists():
        # allow_pickle：cmp_labels 是字符串数组（object dtype）
        return V.verify(module, np.load(out, allow_pickle=True))
    d = convert.build(module, RUNS / module, out_npz=None, excel=False)
    return V.verify(module, d)


def cmd_all(args):
    ensure_dirs()
    for fn, a in ((cmd_prep, argparse.Namespace(check=True)),
                  (cmd_run, args),
```

```
                  (cmd_verify, args)):
        rc = fn(a)
        if rc:
            return rc
    return 0


# ---------------------------------------------------------------------------
def main(argv=None):
    ap = argparse.ArgumentParser(prog='cpp.python.cli', description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter)
    sub = ap.add_subparsers(dest='cmd', required=True)

    sub.add_parser('build').set_defaults(fn=cmd_build)
    sub.add_parser('selfcheck').set_defaults(fn=cmd_selfcheck)

    p = sub.add_parser('prep')
    p.add_argument('--check', action='store_true')
    p.set_defaults(fn=cmd_prep)

    for name, fn in (('run', cmd_run), ('verify', cmd_verify),
                     ('all', cmd_all)):
        p = sub.add_parser(name)
        p.add_argument('q', nargs='*',
                       help='preheat | fullcycle | duration | shrinkage'
                            '（缺省 preheat）')
        if name == 'run':
            p.add_argument('--excel', action='store_true',
                           help='同时生成 result*.xlsx / 表*.csv（写入 data/）')
            p.add_argument('--publish', action='store_true',
                           help='把结果写到 data/<模块>_fields.npz（那是标准结果，'
                                '默认不动；只在确认要更新正式产物时用）')
        p.set_defaults(fn=fn)

    a = ap.parse_args(argv)
    if not hasattr(a, 'check'):
        a.check = False
    if not hasattr(a, 'q'):
        a.q = []
    if not hasattr(a, 'excel'):
        a.excel = False
    if not hasattr(a, 'publish'):
        a.publish = False
    return a.fn(a)


if __name__ == '__main__':
    raise SystemExit(main())
```

```
// ======== 文件：python/gui.py ========
# -*- coding: utf-8 -*-
"""
gui.py —— 核心模拟的图形界面（tkinter + matplotlib）。

启动（从**项目根**，这样 cpp 才是可导入的包）：

    cd H:\\2026 数学建模
    python -m cpp.python.gui

设计要点
--------
* **参数默认值全部取自 defaults.py**，因此界面上的默认设定
  与软件本身的计算口径始终一致。绝不把 31.72 这类常数手抄进控件。
* 求解走子进程 `cpp_sim.exe`，**与 CLI 同一条代码路径**（runner → convert）。
  界面没有独占逻辑 —— 只要路径分开，两边的行为就会逐渐不一致。
* 主线程独占 tkinter；worker 线程只往 queue 里放行；`after(80)` 抽队列。
  重绘另按 500 ms 节流，否则 matplotlib 会吃掉求解器要的 CPU。
* 结果只写到 `runs/<module>/`；要更新 `data/` 里的正式产物须显式勾选"发布"。
"""

import queue
import threading
import time
import traceback
from pathlib import Path
import tkinter as tk
from tkinter import ttk, messagebox, filedialog

# 直接 `python cpp\python\gui.py`（IDE 的 ▶ 就是这么干的）时 __package__ 为空，
# 相对导入会报 "attempted relative import with no known parent package"。
# 这里补上项目根再按模块重跑一遍；被 `-m cpp.python.gui` 正常导入时零开销跳过。
if __name__ == '__main__' and not __package__:
    import runpy
    import sys
    sys.path.insert(0, str(Path(__file__).resolve().parents[2]))
    runpy.run_module('cpp.python.gui', run_name='__main__')
    raise SystemExit(0)

from . import BUILD, CPP, DATA, EXE, RUNS, ensure_dirs
from . import convert, runner, specs
from .gui_plots import LivePlot, ResultPlot, CheckPlot

TITLE = '药材烘干过程数值模拟软件'

# 参数表单：分组 → [(key, 中文标签, 说明)]
PARAM_GROUPS = [
    ('数值', [
        ('NR', '网格数 NR', '节点的径向分辨率；取 20 的整数倍才能零插值输出'),
```

```
        ('dt_start', '启动段 Δt / s', '起始段步长，抹平初始表面通量阶跃'),
        ('dt_main', '主段 Δt / s', '主推进步长'),
        ('dt_start_until', '启动段结束 / s', ''),
        ('dt_fine', '细化段 Δt / s', '终点复核用的细步长'),
        ('picard_tol', 'Picard 容差', '非线性收敛判据（相对）'),
        ('picard_max', 'Picard 上限', ''),
    ]),
    ('边界', [
        ('bc_mode', '边界模式', 'pchip=仅实测段 | pchip_step=硬跳变 | pchip_gp=GPR 外
推'),
        ('bc_t_switch', '两阶段分界 / s', '工况曲线覆盖段终点（= 4 h）'),
        ('bc_width_h', '边界过渡宽度 / h', 'GPR 外推到恒温值的 σ 混合窗'),
        ('bc_T_hold', '恒温段 T∞ / ℃', '实测数据未覆盖时段的显式假设'),
        ('bc_C_hold', '恒温段 C∞ / kg/kg', '同上'),
    ]),
    ('表面（两阶段液膜）', [
        ('surf_on', '启用表面模型', '1 = 启用（默认）'),
        ('surf_eps_preheat', '预热段 ε', '到达表面的水中发生相变的份额（0.03）'),
        ('surf_eps_hold', '恒温段 ε', '液膜在物料之外 ⇒ 物料内无相变源（0）'),
        ('surf_t_ramp', '液膜过渡 / s', '避免 t_switch 处温度瞬间跳降'),
        ('surf_gate', '液膜存在性判据', '1=表面变干后退回 Robin'),
        ('surf_gate_band', '判据带宽', '1.0 ⇒ w=min[1,ratio/2]，带宽必须足够大'),
        ('surf_h_pen', '罚函数系数', '实现 Dirichlet（T_s = 湿球温度）'),
    ]),
    ('物性与几何', [
        ('props_plain', '单套物性', '1 = 单套物性（预热阶段与收缩工况分解用）'),
        ('props_pre', '预热段物性', 'props_a | props_b | props_c'),
        ('props_hold', '恒温段物性', '同上'),
        ('props_win_a_h', '物性过渡起点 / h', ''),
        ('props_win_b_h', '物性过渡终点 / h', ''),
    ]),
    ('终点判据', [
        ('endpoint', '启用终点判据', '1=max C < c_target 时停'),
        ('c_target', '终点含水率', '各点都降到 0.15 以下'),
        ('confirm_steps', '确认步数', '连续 N 步达标才判定'),
        ('refine', '细化复核', '1=从终点前的检查点用细步长复跑'),
        ('refine_window_h', '细化回溯窗口 / h',
         '★ 终点复核从终点前 2.5 h 恢复（早期定为 2 h，实际生效的一直是 2.5 h）'),
    ]),
    ('记录与输出', [
        ('snap_every', '快照间隔 / s', '预热阶段为 1 s；全周期为 3600 s（逐小时）'),
        ('row_every', '行输出间隔 / s', '60 s；预热阶段关闭'),
        ('six_every', '6 小时行间隔 / s', '21600 s；预热与全周期关闭'),
        ('six_min_hour', '6h 行起始 / h',
         '时长预测为 0（含 t=0），收缩工况为 6，两者行数不同'),
```

```
        ('row_include_t0', '行输出含 t=0', '全周期为 1：时长预测与收缩工况为 0'),
        ('trace_every', '守恒追踪间隔 / s', ''),
        ('progress_ms', '进度回报间隔 / ms', '墙钟节流，避免管道成为瓶颈'),
    ]),
]

BOOL_KEYS = {'surf_on', 'surf_gate', 'props_plain', 'endpoint', 'refine',
             'row_include_t0', 'dens_include_t0'}


class App(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title(TITLE)
        self.geometry('1320x900')
        ensure_dirs()

        self.module = tk.StringVar(value='preheat')
        self.status = tk.StringVar(value='就绪')
        self.progress = tk.DoubleVar(value=0.0)
        self.var = {}                  # spec key -> StringVar
        self.q = queue.Queue()
        self.worker = None
        self.run = None
        self.cancelled = False
        self.last_data = None
        self.last_dir = None
        self.t0 = None

        self._build_top()
        self._build_body()
        self._build_log()
        self.load_defaults()
        self.after(80, self._pump)

    # ------------------------------------------------------------------
    # 布局
    # ------------------------------------------------------------------
    def _build_top(self):
        bar = ttk.Frame(self, padding=(10, 8))
        bar.pack(fill='x')
        ttk.Label(bar, text='模块：', font=('', 10, 'bold')).pack(side='left')
        for q, lab in (('preheat', '预热阶段模拟'),
                       ('fullcycle', '全周期干燥模拟'),
                       ('duration', '干燥时长预测'),
                       ('shrinkage', '收缩工况模拟')):
            ttk.Radiobutton(bar, text=lab, value=q, variable=self.module,
                            command=self.on_switch).pack(side='left', padx=4)
        ttk.Separator(bar, orient='vertical').pack(side='left', fill='y',
```

```
                                                padx=10)
        ttk.Label(bar, textvariable=self.status).pack(side='left')

    def _build_body(self):
        body = ttk.Frame(self)
        body.pack(fill='both', expand=True, padx=10)

        left = ttk.Frame(body, width=430)
        left.pack(side='left', fill='y')
        left.pack_propagate(False)
        self._build_params(left)
        self._build_controls(left)

        right = ttk.Notebook(body)
        right.pack(side='left', fill='both', expand=True, padx=(10, 0))
        self.nb = right
        # ⚠ 先建 tab 的 Frame，再用它当 canvas 的 master —— 否则 FigureCanvasTkAgg
        #   会把控件挂到 Notebook 本身而不是 tab 里（repack 也救不回来）。
        self.tabs = {}
        self.live = self.result = self.check = None
        for key, cls, name in (('live', LivePlot, '实时监控'),
                               ('result', ResultPlot, '结果'),
                               ('check', CheckPlot, '守恒')):
            fr = ttk.Frame(right)
            right.add(fr, text=name)
            cv = cls(fr)
            cv.pack(fill='both', expand=True)
            self.tabs[key] = fr
            setattr(self, key, cv)

    def _build_params(self, parent):
        ttk.Label(parent, text='参数（默认值取自 defaults.py）',
                  font=('', 10, 'bold')).pack(anchor='w', pady=(0, 4))
        outer = ttk.Frame(parent)
        outer.pack(fill='both', expand=True)
        cv = tk.Canvas(outer, highlightthickness=0, width=410)
        sb = ttk.Scrollbar(outer, orient='vertical', command=cv.yview)
        cv.configure(yscrollcommand=sb.set)
        sb.pack(side='right', fill='y')
        cv.pack(side='left', fill='both', expand=True)
        inner = ttk.Frame(cv)
        cv.create_window((0, 0), window=inner, anchor='nw')
        inner.bind('<Configure>',
                   lambda e: cv.configure(scrollregion=cv.bbox('all')))
        cv.bind_all('<MouseWheel>',
                    lambda e: cv.yview_scroll(int(-e.delta / 120), 'units'))

        self._entries = {}
        for gname, items in PARAM_GROUPS:
```

```
            lf = ttk.LabelFrame(inner, text=gname, padding=6)
            lf.pack(fill='x', expand=True, pady=3, padx=2)
            for key, label, hint in items:
                row = ttk.Frame(lf)
                row.pack(fill='x', pady=1)
                ttk.Label(row, text=label, width=17, anchor='w').pack(side='left')
                v = tk.StringVar()
                self.var[key] = v
                ent = ttk.Entry(row, textvariable=v, width=18)
                ent.pack(side='left')
                self._entries[key] = ent
                if hint:
                    ttk.Label(row, text='ⓘ', foreground='#3a6ea5',
                              cursor='hand2').pack(side='left', padx=3)
                    self._bind_hint(row.winfo_children()[-1], label, hint)

        adv = ttk.LabelFrame(inner, text='高级（完整 params，可直接编辑）',
                             padding=6)
        adv.pack(fill='both', expand=True, pady=3, padx=2)
        self.adv = tk.Text(adv, height=9, width=48, wrap='none')
        self.adv.pack(fill='both', expand=True)

    def _bind_hint(self, widget, label, hint):
        def show(_e=None):
            messagebox.showinfo(label, hint, parent=self)
        widget.bind('<Button-1>', show)

    def _build_controls(self, parent):
        bar = ttk.Frame(parent, padding=(0, 6))
        bar.pack(fill='x')

        self.btn_build = ttk.Button(bar, text='构建 exe', command=self.do_build)
        self.btn_build.grid(row=0, column=0, padx=2, pady=2, sticky='ew')
        self.btn_prep = ttk.Button(bar, text='准备数据表', command=self.do_prep)
        self.btn_prep.grid(row=0, column=1, padx=2, pady=2, sticky='ew')
        self.btn_run = ttk.Button(bar, text='▶ 运行', command=self.do_run)
        self.btn_run.grid(row=1, column=0, padx=2, pady=2, sticky='ew')
        self.btn_cancel = ttk.Button(bar, text='■ 取消', command=self.do_cancel,
                                     state='disabled')
        self.btn_cancel.grid(row=1, column=1, padx=2, pady=2, sticky='ew')
        self.btn_open = ttk.Button(bar, text='打开输出目录',
                                   command=self.do_open)
        self.btn_open.grid(row=2, column=0, padx=2, pady=2, sticky='ew')
        self.btn_load = ttk.Button(bar, text='载入默认参数',
                                   command=self.load_defaults)
        self.btn_load.grid(row=2, column=1, padx=2, pady=2, sticky='ew')
        bar.columnconfigure(0, weight=1)
        bar.columnconfigure(1, weight=1)
```

```
        self.cb_excel = tk.BooleanVar(value=False)
        self.cb_publish = tk.BooleanVar(value=False)
        self.cb_decomp = tk.BooleanVar(value=True)
        ttk.Checkbutton(parent, text='同时生成 result*.xlsx / 表*.csv（写入 data/）',
                        variable=self.cb_excel).pack(anchor='w')
        ttk.Checkbutton(parent, text='发布到 data/<模块>_fields.npz（会覆盖标准结果）',
                        variable=self.cb_publish).pack(anchor='w')
        ttk.Checkbutton(parent, text='收缩工况：跑效应分解（+约 35 s）',
                        variable=self.cb_decomp).pack(anchor='w')

        ttk.Progressbar(parent, variable=self.progress, maximum=100.0)\
            .pack(fill='x', pady=(6, 2))
        self.lbl_prog = ttk.Label(parent, text='')
        self.lbl_prog.pack(anchor='w')

    def _build_log(self):
        fr = ttk.LabelFrame(self, text='日志', padding=4)
        fr.pack(fill='both', padx=10, pady=(4, 8))
        self.txt = tk.Text(fr, height=9, wrap='none', state='disabled')
        sb = ttk.Scrollbar(fr, orient='vertical', command=self.txt.yview)
        self.txt.configure(yscrollcommand=sb.set)
        sb.pack(side='right', fill='y')
        self.txt.pack(fill='both', expand=True)
        self.txt.tag_configure('err', foreground='#b00020')
        self.txt.tag_configure('ok', foreground='#0b6b3a')

    # ------------------------------------------------------------------
    # 参数
    # ------------------------------------------------------------------
    def load_defaults(self):
        """**唯一**的默认值来源：specs.SPECS（其真值来自 defaults.py）。"""
        module = self.module.get()
        spec = specs.SPECS[module]()
        for k, v in self.var.items():
            if k in spec:
                v.set(str(spec[k]))
        self.adv.delete('1.0', 'end')
        self.adv.insert('1.0', specs.to_params_text(spec, f'runs/{module}'))
        self.log(f'[gui] 已载入「{specs.MODULE_NAMES[module]}」的默认参数')

    def collect(self) -> dict:
        """把表单 + 高级框合成一份 spec。"""
        spec = specs.SPECS[self.module.get()]()
        for k, v in self.var.items():
            if k not in spec:
                continue
            text = v.get().strip()
```

```
            old = spec[k]
            try:
                if isinstance(old, bool) or k in BOOL_KEYS:
                    spec[k] = int(float(text)) != 0
                elif isinstance(old, int):
                    spec[k] = int(float(text))
                elif isinstance(old, float):
                    spec[k] = float(text)
                else:
                    spec[k] = text
            except ValueError:
                raise ValueError(f'参数 {k} 的值不合法：{text!r}') from None
        # 高级框里显式给出的键优先
        for line in self.adv.get('1.0', 'end').splitlines():
            line = line.strip()
            if not line or line[0] == '#' or '=' not in line:
                continue
            k, val = (s.strip() for s in line.split('=', 1))
            if k in ('outdir',):
                continue
            if k in spec and isinstance(spec[k], (int, float, bool)):
                spec[k] = type(spec[k])(float(val)) if not isinstance(
                    spec[k], str) else val
            else:
                spec[k] = val
        return spec

    def on_switch(self):
        self.load_defaults()

    def select_tab(self, key):
        """切到 'live' / 'result' / 'check' 三页之一（截图与演示脚本用）。"""
        if key in self.tabs:
            self.nb.select(self.tabs[key])

    # ------------------------------------------------------------------
    # 动作
    # ------------------------------------------------------------------
    def do_build(self):
        self.log('[build] 正在构建 cpp_sim.exe …')

        def work():
            import subprocess
            r = subprocess.run(
                ['powershell', '-NoProfile', '-ExecutionPolicy', 'Bypass',
                 '-File', str(CPP / 'build.ps1')],
                cwd=str(CPP), capture_output=True, text=True,
                encoding='utf-8', errors='replace')
            for ln in (r.stdout or '').splitlines():
                self.q.put(ln)
```

```
            self.q.put(('__exit__', r.returncode, 0.0))
        threading.Thread(target=work, daemon=True).start()

    def do_prep(self):
        self.log('[prep] 正在生成 tables/（附件拟合 + GPR 预算表）…')

        def work():
            try:
                from . import prep_tables
                prep_tables.write_attach1(verbose=False)
                prep_tables.write_gpr(verbose=False)
                prep_tables.write_radius(verbose=False)
                self.q.put('[prep] tables/ 已生成')
                self.q.put(('__exit__', 0, 0.0))
            except Exception as e:                # noqa: BLE001
                self.q.put(f'ERR msg={type(e).__name__}: {e}')
                self.q.put(('__exit__', 1, 0.0))
        threading.Thread(target=work, daemon=True).start()

    def do_run(self):
        try:
            spec = self.collect()
        except ValueError as e:
            messagebox.showerror('参数错误', str(e), parent=self)
            return
        module = self.module.get()
        decomp = self.cb_decomp.get() and module == 'shrinkage'
        excel = self.cb_excel.get()
        publish = self.cb_publish.get()

        self.cancelled = False
        self.btn_run.configure(state='disabled')
        self.btn_cancel.configure(state='normal')
        self.progress.set(0.0)
        self.live.reset()
        self.live.force_draw()
        self.t0 = time.time()
        self.log('=' * 68)
        self.log(f'[{module}] 开始求解  NR={spec.get("NR")}  '
                 f't_end={spec.get("t_end")} s  geom={spec.get("geom")}')

        def work():
            try:
                run = runner.AsyncRun(spec).start()
                self.run = run
                while True:
                    item = run.q.get()
                    self.q.put(item)
                    if isinstance(item, tuple) and item[0] == '__exit__':
                        break
```

```
                if run.returncode != 0:
                    return
                if self.cancelled:
                    return
                self.q.put('[gui] 组装产物…')
                kw = {}
                if decomp:
                    from .cli import effect_decomposition, _duration_base_h
                    vals = []
                    for sp, tag in ((specs.spec_shrinkage_case_props(), '① 固定 R +
物性方案 C'),
                                    (specs.spec_shrinkage_case_geom(),
                                     '② 收缩 R(t) + 物性方案 B')):
                        r2 = runner.AsyncRun(sp).start()
                        while True:
                            it = r2.q.get()
                            if isinstance(it, tuple) and it[0] == '__exit__':
                                break
                            self.q.put(it)
                        s = convert._read_summary(r2.run_dir)
                        vals.append(float(s.get('t_end', float('nan'))) / 3600.0)
                        self.q.put(f'[{tag}] t_end = {vals[-1]:.4f} h')
                    kw = dict(cmp_labels=['① 固定 R + 物性方案 C（分离物性效应）',
                                          '② 收缩 R(t) + 物性方案 B（分离几何效应）',
                                          '③ 收缩+物性方案 C'],
                              cmp_vals=vals + [float('nan')],
                              q3_base_h=_duration_base_h())
                out_npz = run.run_dir / 'out.npz'
                d = convert.build(module, run.run_dir, out_npz=out_npz,
                                  excel=excel, **kw)
                if decomp:
                    d['cmp_vals'][2] = float(d['t_end']) / 3600.0
                    import numpy as np
                    np.savez(out_npz, **d)
                if publish:
                    import numpy as np
                    from common import config as cfg
                    np.savez(cfg.DIR_DATA / f'{module}_fields.npz', **d)
                    self.q.put(f'[publish] 已更新 data/{module}_fields.npz')
                self.q.put(('__done__', module, run.run_dir, d))
            except Exception as e:               # noqa: BLE001
                self.q.put(f'ERR msg={type(e).__name__}: {e}')
                self.q.put(f'[traceback] {traceback.format_exc()}')
                self.q.put(('__exit__', 1, 0.0))
        threading.Thread(target=work, daemon=True).start()

    def do_cancel(self):
```

```
        self.cancelled = True
        if self.run is not None:
            self.run.cancel()
        self.log('[gui] 已请求取消')

    def do_open(self):
        import os
        d = RUNS / self.module.get()
        d.mkdir(parents=True, exist_ok=True)
        os.startfile(str(d))                      # noqa: S606  (Windows)

    # ------------------------------------------------------------------
    # 队列抽取
    # ------------------------------------------------------------------
    def _pump(self):
        n = 0
        while n < 200:
            try:
                item = self.q.get_nowait()
            except queue.Empty:
                break
            n += 1
            self._handle(item)
        self.after(80, self._pump)

    def _handle(self, item):
        if isinstance(item, tuple):
            kind = item[0]
            if kind == '__exit__':
                rc, secs = item[1], item[2]
                self.btn_run.configure(state='normal')
                self.btn_cancel.configure(state='disabled')
                self.status.set(f'退出码 {rc}   用时 {secs:.1f} s')
                if rc == 0:
                    self.progress.set(100.0)
            elif kind == '__done__':
                _, module, run_dir, d = item
                self.last_data, self.last_dir = d, run_dir
                self._show_result(module, d, run_dir)
            return

        line = str(item)
        self.log(line)
        d = runner.parse_line(line)
        if not d:
            return
        if d['kind'] == 'PROG':
            self.live.add(d)
            self.live.maybe_draw(500)
        elif d['kind'] == 'frac':
```

```
            self.progress.set(min(100.0, d['frac'] * 100.0))
            el = time.time() - (self.t0 or time.time())
            f = max(d['frac'], 1e-6)
            self.lbl_prog.config(
                text=f'{d["frac"]*100:5.1f}%   已用 {el:6.1f} s   '
                     f'预计剩余 {max(el/f - el, 0):6.1f} s')
        elif d['kind'] == 'ERR':
            self.status.set('出错：' + str(d.get('msg', '')))
        elif d['kind'] == 'READY':
            self.status.set(f'求解中 run_id={d.get("run_id")}')
        elif d['kind'] == 'STAGE':
            self.status.set(f'阶段：{d.get("name")}')
        elif d['kind'] == 'DONE':
            self.status.set(
                f'完成  steps={d.get("steps")}  '
                f'质量失衡={d.get("mass_rel")}  能量失衡={d.get("energy_rel")}')

    def _show_result(self, module, d, run_dir):
        s = convert._read_summary(run_dir)
        self.result.show(module, d, field='C')
        try:
            self.check.show(module, Path(run_dir), s)
        except Exception:                        # noqa: BLE001
            pass
        if module == 'preheat':
            self.log(f'[结果] 表面温度 {d["T"][-1][-1]-273.15:.4f} ℃   '
                     f'表面含水率 {d["C"][-1][-1]:.4f}')
        else:
            h = float(d["t_end"]) / 3600.0
            mr, er = s.get('mass_rel'), s.get('energy_rel')
            self.log(f'[结果] t_end = {h:.4f} h = {h/24:.4f} 天   '
                     f'质量失衡 {mr:.3e}   能量失衡 {er:.3e}')
        self.status.set('完成')

    def log(self, text):
        self.txt.configure(state='normal')
        tag = ''
        if text.startswith('ERR') or 'Error' in text or 'FAIL' in text:
            tag = 'err'
        elif text.startswith('[结果]') or text.startswith('DONE'):
            tag = 'ok'
        self.txt.insert('end', text + '\n', tag)
        self.txt.see('end')
        self.txt.configure(state='disabled')


def main():
    app = App()
    app.mainloop()
    return 0
```

```
if __name__ == '__main__':
    raise SystemExit(main())

// ======== 文件：python/gui_plots.py ========
# -*- coding: utf-8 -*-
"""
gui_plots.py —— 界面里的三个 matplotlib 画布。

  Live   实时监控：由 exe 的 PROG 行驱动（中心/表面含水率、表面温度 ~ 时间）
  Result 结果：场演化热图（T / C）+ 干燥曲线
  Check  守恒：mass_rel / energy_rel 轨迹

⚠ 重绘必须**节流**：按 200 ms 的 PROG 节奏重绘 matplotlib 会吃掉求解器需要的
  CPU。这里只在 `interval_ms` 到点、且数据确有增长时才 `draw_idle()`。
"""

import time
from pathlib import Path

import numpy as np
import matplotlib
matplotlib.use('TkAgg')                          # 必须：嵌入式画布
from matplotlib.backends.backend_tkagg import (        # noqa: E402
    FigureCanvasTkAgg, NavigationToolbar2Tk)
from matplotlib.figure import Figure                    # noqa: E402


class BaseCanvas:
    """带工具栏的 matplotlib 画布基类（中文标签用系统黑体）。"""

    def __init__(self, master):
        matplotlib.rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei',
                                                  'DejaVu Sans']
        matplotlib.rcParams['axes.unicode_minus'] = False
        self.fig = Figure(figsize=(8, 4.2), dpi=100, constrained_layout=True)
        self.canvas = FigureCanvasTkAgg(self.fig, master=master)
        self.widget = self.canvas.get_tk_widget()
        self.toolbar = NavigationToolbar2Tk(self.canvas, master,
                                            pack_toolbar=False)
        self._last = 0.0
        self._dirty = False

    def pack(self, **kw):
        self.toolbar.pack(side='bottom', fill='x')
        self.widget.pack(**kw)

    def touch(self):
        self._dirty = True
```

```
    def maybe_draw(self, interval_ms=500):
        if not self._dirty:
            return
        now = time.time() * 1000.0
        if now - self._last < interval_ms:
            return
        self._last = now
        self._dirty = False
        self.canvas.draw_idle()

    def force_draw(self):
        self._last = time.time() * 1000.0
        self._dirty = False
        self.canvas.draw()


# ============================================================================
class LivePlot(BaseCanvas):
    """实时监控：随 PROG 行累积。"""

    def __init__(self, master):
        super().__init__(master)
        self.reset()

    def reset(self):
        self.t_h, self.cc, self.cs, self.ts, self.R = [], [], [], [], []
        self.fig.clear()
        gs = self.fig.add_gridspec(1, 2)
        self.ax1 = self.fig.add_subplot(gs[0, 0])
        self.ax2 = self.fig.add_subplot(gs[0, 1])
        self.ax1.set_xlabel('时间 / h')
        self.ax1.set_ylabel('含水率 / (kg/kg)')
        self.ax1.set_title('含水率历程')
        self.ax2.set_xlabel('时间 / h')
        self.ax2.set_ylabel('表面温度 / ℃')
        self.ax2.set_title('表面温度')
        for ax in (self.ax1, self.ax2):
            ax.grid(alpha=0.3)
        self._ln = {}
        self.touch()

    def add(self, d):
        """d 是解析后的 PROG 字典。"""
        if 't_h' not in d:
            return
        self.t_h.append(d['t_h'])
        self.cc.append(d.get('Cc', np.nan))
        self.cs.append(d.get('Cs', np.nan))
        self.ts.append(d.get('Ts', np.nan))
```

```
        if 'R' in d:
            self.R.append(d['R'])
        self._redraw()

    def _redraw(self):
        if not self.t_h:
            return
        x = np.asarray(self.t_h)
        self.ax1.clear()
        self.ax2.clear()
        self.ax1.plot(x, self.cc, lw=1.4, label='中心')
        self.ax1.plot(x, self.cs, lw=1.4, label='表面')
        self.ax1.axhline(0.15, color='crimson', ls='--', lw=1,
                         label='终点判据 0.15')
        self.ax1.set_xlabel('时间 / h')
        self.ax1.set_ylabel('含水率 / (kg/kg)')
        self.ax1.set_title('含水率历程')
        self.ax1.grid(alpha=0.3)
        self.ax1.legend(loc='best', fontsize=8)
        self.ax2.plot(x, self.ts, lw=1.4, color='darkorange')
        self.ax2.axhline(41.30, color='steelblue', ls=':', lw=1,
                         label='湿球 41.30 ℃')
        self.ax2.set_xlabel('时间 / h')
        self.ax2.set_ylabel('表面温度 / ℃')
        self.ax2.set_title('表面温度')
        self.ax2.grid(alpha=0.3)
        self.ax2.legend(loc='best', fontsize=8)
        self.touch()


# ============================================================================
class ResultPlot(BaseCanvas):
    """结果：场演化热图 + 干燥曲线。"""

    def __init__(self, master):
        super().__init__(master)

    def reset(self):
        self.fig.clear()
        gs = self.fig.add_gridspec(1, 2, width_ratios=[1.15, 1])
        self.axF = self.fig.add_subplot(gs[0, 0])
        self.axD = self.fig.add_subplot(gs[0, 1])
        self.touch()

    def show(self, module, d, field='C'):
        """d 是 convert.build 的返回 dict。"""
        self.fig.clear()
        gs = self.fig.add_gridspec(1, 2, width_ratios=[1.15, 1])
        axF = self.fig.add_subplot(gs[0, 0])
        axD = self.fig.add_subplot(gs[0, 1])
```

```
        # ---- 场演化热图 ----
        moving = 'xi' in d and 'r' not in d
        if moving:
            x = np.asarray(d['xi'], float)
            xlab = 'ξ = r/R(t)'
        else:
            x = np.asarray(d['r'], float) * 100.0
            xlab = '距离 / cm'
        key = 'snap' if ('snap' in d and len(d['snap']['t'])) else 'row'
        if key == 'snap':
            tt, ff = d['snap']['t'], d['snap'][field]
        else:
            tt = d.get('row_t', d.get('t'))
            ff = d.get('row_' + field, d.get(field))
        if tt is None or len(tt) == 0:
            axF.text(0.5, 0.5, '本次运行没有场快照\n（快照间隔 snap_every = 0）',
                     ha='center', va='center', transform=axF.transAxes)
        else:
            t_h = np.asarray(tt, float) / 3600.0
            Z = np.asarray(ff, float)
            if field == 'T':
                Z = Z - 273.15
                lab, cmap = '温度 / ℃', 'inferno'
            else:
                lab, cmap = '含水率 / (kg/kg)', 'viridis_r'
            im = axF.pcolormesh(t_h, x, Z.T, shading='auto', cmap=cmap)
            self.fig.colorbar(im, ax=axF, label=lab, pad=0.02)
            if t_h.max() > 4.0:
                axF.axvline(4.0, color='w', ls='--', lw=1, alpha=0.8)
            axF.set_xlabel('时间 / h')
            axF.set_ylabel(xlab)
            axF.set_title(f'{module.upper()}  {"温度" if field=="T" else "水分"}场演
化')

        # ---- 干燥曲线 ----
        if 'row_t' in d and len(d.get('row_t', [])):
            th = np.asarray(d['row_t'], float) / 3600.0
            rc = np.asarray(d['row_C'], float)
            axD.plot(th, rc[:, 0], lw=1.6, label='中心')
            axD.plot(th, rc[:, -1], lw=1.6, label='表面')
            axD.axhline(0.15, color='crimson', ls='--', lw=1, label='0.15')
            axD.set_xlabel('时间 / h')
            axD.set_ylabel('含水率 / (kg/kg)')
            axD.set_title('干燥曲线')
            axD.grid(alpha=0.3)
            axD.legend(loc='best', fontsize=8)
        elif 'dens_t' in d and len(d.get('dens_t', [])):
            th = np.asarray(d['dens_t'], float) / 3600.0
```

```
            dc = np.asarray(d['dens_C'], float)
            axD.plot(th, dc[:, 0], lw=1.6, label='中心')
            axD.plot(th, dc[:, 1], lw=1.6, label='表面')
            axD.axhline(0.15, color='crimson', ls='--', lw=1)
            axD.set_xlabel('时间 / h')
            axD.set_ylabel('含水率 / (kg/kg)')
            axD.set_title('干燥曲线')
            axD.grid(alpha=0.3)
            axD.legend(loc='best', fontsize=8)
        else:
            axD.text(0.5, 0.5, '没有逐时序列', ha='center', va='center',
                     transform=axD.transAxes)
        self.force_draw()


# ============================================================================
class CheckPlot(BaseCanvas):
    """守恒：逐步追踪的相对失衡 + 收支曲线。"""

    def __init__(self, master):
        super().__init__(master)

    @staticmethod
    def _read(path, min_cols):
        """读 CSV；空文件/列数不足都返回 None（**不报警告**）。"""
        p = Path(path)
        if not p.exists() or p.stat().st_size == 0:
            return None
        try:
            a = np.genfromtxt(p, delimiter=',', comments='#', skip_header=1)
        except Exception:                          # noqa: BLE001
            return None
        if a is None:
            return None
        a = np.atleast_2d(a)
        if a.size == 0 or a.shape[0] == 0 or a.shape[1] < min_cols:
            return None
        return a

    def show(self, module, run_dir, summary):
        self.fig.clear()
        gs = self.fig.add_gridspec(1, 2)
        ax1 = self.fig.add_subplot(gs[0, 0])
        ax2 = self.fig.add_subplot(gs[0, 1])

        b = self._read(Path(run_dir) / 'budget.csv', 3)
        if b is not None:
            ax1.semilogy(b[:, 0] / 3600.0, np.maximum(b[:, 1], 1e-18),
                         lw=1.3, label='质量恒等式')
            ax1.semilogy(b[:, 0] / 3600.0, np.maximum(b[:, 2], 1e-18),
```

```
                        lw=1.3, label='能量恒等式')
            ax1.axhline(1e-9, color='crimson', ls='--', lw=1, label='门槛 1e-9')
            ax1.set_yscale('log')
        else:
            # 预热阶段模块 只有 1800 s，逐小时追踪点全落在区间之外 ⇒ 序列为空。
            # 这是**对的**（该模块本身不启用逐小时追踪），不是缺数据。
            ax1.text(0.5, 0.5,
                     '本次运行没有逐小时追踪点\n'
                     '（预热阶段模块 时长 1800 s < 1 h；\n'
                     '  或 trace_every 大于总时长）',
                     ha='center', va='center', fontsize=9,
                     transform=ax1.transAxes)
        ax1.set_xlabel('时间 / h')
        ax1.set_ylabel('相对失衡')
        ax1.set_title('逐步守恒残差（与求解器同一离散式）')
        ax1.grid(alpha=0.3, which='both')
        if ax1.get_legend_handles_labels()[0]:
            ax1.legend(loc='best', fontsize=8)

        m = self._read(Path(run_dir) / 'budget_mass.csv', 4)
        if m is not None:
            ax2.plot(m[:, 0] / 3600.0, m[:, 1], lw=1.4, label='ΔM 存量变化')
            ax2.plot(m[:, 0] / 3600.0, m[:, 2], lw=1.4, label='−∫表面传质')
            ax2.plot(m[:, 0] / 3600.0, m[:, 3], lw=1.4, label='∫边界内缩携带')
            ax2.legend(loc='best', fontsize=8)
        else:
            ax2.text(0.5, 0.5,
                     '质量收支分解仅移动边界（收缩工况模块）产出\n'
                     '（固定域没有"边界内缩携带"这一项）',
                     ha='center', va='center', fontsize=9,
                     transform=ax2.transAxes)
        ax2.set_xlabel('时间 / h')
        ax2.set_title('质量收支')
        ax2.grid(alpha=0.3)
        self.force_draw()

// ======== 文件：build.ps1 ========
<#
  build.ps1 —— 构建 cpp_sim.exe（直接调 MinGW g++，无第三方依赖）。

  保真相关的编译标志（每一条都影响与 Python 参考实现的贴合度）：

    -O2                 内层是 81 元素的内存受限标量运算，-O3 收益极小，
                        却增加编译器重结合归约的机会。
    -ffp-contract=off   GCC 默认 -ffp-contract=fast 会把 a*b+c 融成 FMA，
                        而 numpy 的 ufunc 路径不会 —— 这会在 cap/dt*u + 0.5*F
                        这类表达式上引入系统性 ~1e-16 相对偏差。
    不用 -march=native  会启用 FMA/AVX 并使二进制依赖机器；本题以 exp 为主，
                        收益极小，可复现性更重要。
    绝不用 -ffast-math  / -Ofast 会破坏 NaN 语义与重结合。移动网格代码里有显式
```

```
                        的 NaN 防护，守恒恒等式也依赖 CN 梯形求值的精确性 ——
                        开了会让 mass_rel/energy_rel < 1e-9 的门槛静默失效。
    不定义 -DNDEBUG      Python 侧依赖活断言（check_grid / build_xi_grid /
                        Snapshots），本工程把它们移植为始终生效的检查。
    -municode           用 wmain 拿宽字符命令行，规避中文路径的 ANSI 解码问题。
    -static...          让 exe 能从任意 cwd 运行，不依赖 PATH 里有 MinGW。
#>
param(
    [switch]$NoSelfCheck,   # 跳过构建后的自检
    [switch]$Clean          # 先清空 build/
)

$ErrorActionPreference = 'Stop'
$root = $PSScriptRoot

# 挑一个**真能编译**的 g++，而不只是"存在"。
# ⚠ 本机 F:\mingw64 的 g++ 15.1.0 是坏的：g++.exe --version 正常，但 cc1plus.exe
#   加载失败（0xC0000135 STATUS_DLL_NOT_FOUND，缺 DLL），编译静默退出 1 且不报错。
#   所以这里实际编译一个空文件来验证，Qt 自带的 MinGW 13.1.0 可用。
function Test-Gxx([string]$p) {
    if (-not (Test-Path $p)) { return $false }
    $tmp = Join-Path $env:TEMP ("probe_" + [guid]::NewGuid().ToString('N'))
    New-Item -ItemType Directory -Force $tmp | Out-Null
    Set-Content -Path "$tmp\p.cpp" -Value 'int main(){return 0;}' -Encoding ascii
    $null = & $p -c "$tmp\p.cpp" -o "$tmp\p.o" 2>&1
    $ok = (Test-Path "$tmp\p.o")
    Remove-Item $tmp -Recurse -Force -ErrorAction SilentlyContinue
    return $ok
}

$gxx = $null
foreach ($cand in @('F:\Qt\Tools\mingw1310_64\bin\g++.exe',
                    'F:\mingw64\bin\g++.exe',
                    'F:\Qt\Tools\mingw1310_64\bin\g++.exe')) {
    if (Test-Gxx $cand) { $gxx = $cand; break }
}
if (-not $gxx) { throw "找不到可用的 g++（试过 F:\Qt\Tools\mingw1310_64 与 F:\mingw64）
" }
Write-Host "==> 使用编译器 $gxx" -ForegroundColor DarkGray

$outDir = Join-Path $root 'build'
if ($Clean -and (Test-Path $outDir)) { Remove-Item $outDir -Recurse -Force }
New-Item -ItemType Directory -Force $outDir | Out-Null

$src = Get-ChildItem (Join-Path $root 'src\*.cpp') | ForEach-Object { $_.FullName }
if ($src.Count -eq 0) { throw "src\ 下没有 .cpp" }
```

```
$flags = @(
    '-std=c++17', '-O2', '-Wall', '-Wextra', '-Wpedantic',
    '-ffp-contract=off', '-fno-fast-math', '-fno-math-errno',
    '-static', '-static-libgcc', '-static-libstdc++',
    '-municode',
    '-I' + (Join-Path $root 'include')
)
$exe = Join-Path $outDir 'cpp_sim.exe'

Write-Host "==> 编译 $($src.Count) 个源文件 -> $exe" -ForegroundColor Cyan
& $gxx @flags -o $exe @src
if ($LASTEXITCODE -ne 0) { throw "编译失败（退出码 $LASTEXITCODE）" }

$size = [math]::Round((Get-Item $exe).Length / 1MB, 2)
Write-Host "==> 构建成功（$size MB）" -ForegroundColor Green

if (-not $NoSelfCheck) {
    Write-Host "==> 运行自检" -ForegroundColor Cyan
    Push-Location $root
    try { & $exe --selfcheck } finally { Pop-Location }
    if ($LASTEXITCODE -ne 0) {
        Write-Host "自检存在失败项（退出码 $LASTEXITCODE）" -ForegroundColor Yellow
        exit $LASTEXITCODE
    }
}

// ======== 文件：CMakeLists.txt ========
# CMakeLists.txt —— 次要构建方式（给 IDE / VS2022 用）。
#
# 主路径是 `build.ps1`：本工程 12 个源文件、零第三方依赖，直接调 g++ 更快也更好
# 诊断。这里只为了在 CLion / VS 里能一键构建。
#
# ⚠ 保真相关的编译标志必须与 build.ps1 **逐条一致**，否则数值会与 Python 参照
#   产生系统性偏差（详见 README「构建标志」一节）。
cmake_minimum_required(VERSION 3.16)
project(cpp_sim CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release)
endif()

add_executable(cpp_sim
    src/interp.cpp
    src/params.cpp
    src/grid.cpp
```

```
    src/props.cpp
    src/psychro.cpp
    src/transition.cpp
    src/solver.cpp
    src/budget.cpp
    src/moving.cpp
    src/run.cpp
    src/selfcheck.cpp
    src/main.cpp
)

target_include_directories(cpp_sim PRIVATE include)

# -O2（不是 -O3）、-ffp-contract=off、绝不 -ffast-math、不定义 NDEBUG。
target_compile_options(cpp_sim PRIVATE
    -O2 -Wall -Wextra -Wpedantic
    -ffp-contract=off -fno-fast-math -fno-math-errno
)
if(MINGW)
    # -municode：wmain 拿宽字符命令行，规避中文路径的 ANSI 解码问题
    target_compile_options(cpp_sim PRIVATE -municode)
    target_link_options(cpp_sim PRIVATE
        -static -static-libgcc -static-libstdc++ -municode)
endif()
```